МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
Гомельский государственный технический университет имени П.О.Сухого

Факультет автоматизированных и информационных систем

Кафедра «Информационные технологии»

специальность 6-05-0611-01 «Информационные системы и технологии»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе

по дисциплине «Распределенные информационные системы»

на тему: Распределенная система для решения СЛАУ методом Гаусса

Исполнитель: студент группы ИТП-31
Мартынов В.Ю.
Руководитель: доцент
Комраков В.В.

Дата проверки:_____

Дата допуска к защите: _____

Оценка работы: _____

Подписи членов комиссии по защите курсового проекта:

_____

Гомель 2026

СОДЕРЖАНИЕ

Введение 5
1 Аналитический обзор методов и средств решения поставленной задачи 6
1.1 Выбор метода решения СЛАУ 6
1.2 Выбор архитектуры распределенной системы 8
1.3 Выбор средств реализации распределенной системы 11
2 Алгоритмический анализ задачи 14
2.1 Архитектурные особенности реализации методов решения СЛАУ .. 14
2.2 Последовательный алгоритм решения СЛАУ 15
2.3 Распределенный алгоритм решения СЛАУ 17
3 Разработка, тестирование и верификация распределенной системы 22
3.1 Состав программных модулей и структура классов приложения 22
3.2 Реализация вычислительного ядра и алгоритмов решения 23
3.3 Реализация пользовательского интерфейса 24
3.4 Тестирование и верификация 25
Заключение 31
Список использованных источников 32
Приложение А – Исходный код приложения 33
Приложение Б – Экранные копии графического интерфейса приложения 67
Приложение В – Листинг модульных тестов 69
Приложение Г – Результаты верификации приложения 94

ВВЕДЕНИЕ

Современные задачи в науке, технике и анализе больших данных всё чаще требуют многократного решения систем линейных алгебраических уравнений (СЛАУ) очень высокой размерности – от сотен тысяч до миллионов неизвестных. Такие системы возникают при численном моделировании физических процессов (гидродинамика, механика деформируемых тел, электромагнетизм), задачах оптимизации и машинного обучения, обработке больших графов и сетей, геофизике, обработке изображений и сигналов, а также в многошаговых схемах интегрирования и задачах с множественными правыми частями. Последовательное выполнение классических алгоритмов на одной вычислительной машине в большинстве случаев становится невозможным из-за большого объёма требуемой памяти и длительного времени расчёта. Актуальность разработки эффективных распределённых систем решения таких СЛАУ определяется именно необходимостью многократного и быстрого решения больших систем в реальных приложениях.

В этих условиях основным инструментом становятся распределённые и параллельные вычислительные системы. Распределение данных и вычислительной нагрузки между множеством процессоров позволяет повысить производительность и обрабатывать задачи, недоступные однопроцессорным и даже мощным многоядерным системам.

Целью курсовой работы является разработка распределённой системы для решения систем линейных алгебраических уравнений с высокой размерностью неизвестных и проведение сравнительного анализа её производительности с последовательным и многопоточным вариантами.

Задачи курсового проекта заключаются в следующем:

– провести анализ источников и программного обеспечения для решения систем линейных алгебраических уравнений большой размерности;

– описать существующие подходы, методы и используемые библиотеки;

– определить перечень исходных данных и требуемых результатов;

– разработать обобщённую функциональную схему приложения, спроектировать иерархию классов и описать их взаимодействие;

– реализовать разработанные алгоритмы в виде программы;

– описать процесс тестирования созданных классов и модулей, выполнить верификацию приложения;

– обобщить полученные результаты, отметить достоинства и недостатки разработанного решения, указать практическую значимость работы.

Разработка такой системы позволит количественно оценить выигрыш от распределения вычислений при решении больших СЛАУ и выявить ключевые ограничения, связанные с коммуникациями и масштабируемостью алгоритма, что имеет практическое значение для современных научных и инженерных приложений, требующих многократного решения систем большой размерности.

1 АНАЛИТИЧЕСКИЙ ОБЗОР МЕТОДОВ И СРЕДСТВ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ

1.1 Выбор метода решения СЛАУ

1.1.1. Основными методами решения СЛАУ являются прямые методы. Линейная алгебра изучает векторные пространства, отображения и связанные с ними объекты – матрицы и определители, которые служат фундаментальными инструментами для работы с СЛАУ. Матрица представляет собой прямоугольную таблицу чисел, которая может описывать линейное преобразование. Система из n уравнений с n неизвестными записывается в матричной форме следующим образом (формула 1.1):

$$Ax = b \tag{1.1}$$

где $A \in R$ – матрица коэффициентов, $x \in R$ – вектор неизвестных, $b \in R$ – вектор правой части.

Все численные методы решения систем линейных алгебраических уравнений (СЛАУ) традиционно делят на две основные группы: прямые методы и итерационные методы. Эти группы различаются по принципу получения решения, объёму потребляемой памяти, устойчивости к ошибкам округления и типам задач, для которых они наиболее эффективны.

Прямые методы дают точное решение за один проход алгоритма, причём количество арифметических операций заранее известно и зависит только от размерности системы n – для плотных матриц обычно порядка $O(n^3)$ [1]. К наиболее распространённым прямым методам относятся классический метод Гаусса, метод Гаусса-Жордана, LU-разложение, разложение Холецкого, метод прогонки.

Классический метод Гаусса остаётся одним из самых простых и понятных алгоритмов. Он состоит из этапа прямого хода (исключение переменных с приведением матрицы к верхнетреугольному виду) и этапа обратной подстановки. Частичный выбор ведущего элемента значительно улучшает численную устойчивость. Главные достоинства метода – минимализм (не требуется хранить дополнительные матрицы), отсутствие настраиваемых параметров и прозрачные зависимости по данным, что делает его относительно удобным для параллельной реализации. К недостаткам относятся высокая вычислительная сложность $O(n^3)$ и $O(n^2)$ памяти для плотных систем, а также чувствительность к плохой обусловленности матрицы, когда ошибки округления могут сильно накапливаться.

Метод Гаусса-Жордана развивает идею Гаусса, но продолжает исключение и над диагональю, приводя матрицу к единичной форме. Это удобно, если требуется сразу получить обратную матрицу или решить систему для нескольких правых частей без повторного исключения. Однако алгоритм требует примерно в 1,5-2 раза больше операций, чем стандартный метод Гаусса.

LU-разложение представляет собой факторизацию матрицы $A = LU$ (нижняя треугольная L с единичной диагональю и верхняя треугольная U), после чего решение системы сводится к двум последовательным подстановкам: сначала $Ly = b$, затем $Ux = y$. С частичным выбором ведущего элемента ($PA = LU$, где P – матрица перестановок) метод становится устойчивым для большинства практических задач. Главное преимущество *LU* – возможность многократного использования разложения для разных правых частей b при фиксированной матрице A (каждое новое решение требует всего $O(n^2)$ операций). Недостатки – дополнительная память для хранения L и U и чуть большая сложность кода по сравнению с простым Гауссом.

Разложение Холецкого применимо только к симметричным положительно определённым матрицам. Оно примерно в 2 раза быстрее обычного *LU*-разложения, требует вдвое меньше памяти (хранится только нижний треугольник L) и обладает численной устойчивостью. Метод применим для задач, где матрица системы гарантированно симметрична и положительно определённа. Однако метод неприменим, если матрица не удовлетворяет условиям симметрии и положительной определённости.

Метод прогонки сводит решение к двум проходам: прямому (вычисление прогоночных коэффициентов) и обратному (нахождение неизвестных). Сложность линейная $O(n)$, память тоже $O(n)$, что делает его эффективным для задач конечно-разностной аппроксимации одномерных и двумерных уравнений в частных производных. Минус – строгая ограниченность классом матриц с ненулевыми элементами только на главной и прилегающих диагоналях.

1.1.2. Итерационные методы начинают с начального приближения и последовательно уменьшают невязку на каждой итерации [2]. К ним относятся метод Якоби, Гаусса-Зейделя, метод последовательной верхней релаксации, а также современные методы: сопряжённых градиентов, *GMRES*, *BiCGStab* и т.д. Они эффективны для очень больших разреженных систем, так как требуют гораздо меньше памяти, позволяют останавливать процесс, как только норма невязки или относительная ошибка достигает заданного уровня.

Общий вид большинства стационарных итерационных методов можно записать в форме (формула 1.2):

$$x^{(k+1)} = x^{(k)} + M^{-1}(b - Ax^{(k)}) \tag{1.2}$$

где M – матрица расщепления, $(b - Ax^{(k)})$ – невязка на k-й итерации.

Метод Якоби на каждой итерации вычисляет приближение для каждой переменной, используя только значения из предыдущей итерации. Благодаря этому метод легко распараллеливается. Основной недостаток – медленная сходимость, которая требует строгого диагонального преобладания матрицы.

Метод Гаусса-Зейделя развивает идею Якоби, но при вычислении новой компоненты уже используются значения, полученные на текущей итерации для предыдущих переменных. Это ускоряет сходимость примерно в 2-3 раза. Однако последовательное обновление компонент создаёт зависимости, из-за чего метод плохо поддаётся параллелизации: приходится вычислять переменные строго по порядку.

Метод сопряжённых градиентов (*CG*) на каждой итерации минимизирует квадратичную функциональную форму, связанную с *A*-нормой ошибки. *CG* требует хранения всего нескольких векторов (примерно 4-5 векторов размера n) и одного умножения матрицы на вектор за итерацию. При хорошей обусловленности число итераций пропорционально квадратному корню из числа обусловленности матрицы. Главное ограничение – применимость только к симметричным положительно определённым системам.

GMRES (обобщённый метод минимальных остатков) минимизирует норму невязки в подпространстве Крылова возрастающей размерности, строя ортогональную базу с помощью процесса Арнольди. *GMRES* гарантированно уменьшает невязку на каждой итерации и в точной арифметике сходится максимум за n шагов. На практике чаще используют вариант с перезапуском *GMRES*(m), где m – фиксированный размер подпространства (обычно 20-100), чтобы ограничить потребление памяти и вычислений. Недостатки – рост затрат на ортогонализацию и хранение базы при большом m, а также необходимость периодического перезапуска.

BiCGStab (стабилизированный метод бисопряжённых градиентов) требует постоянного (не растущего) числа векторов (около 8), не нуждается в перезапуске и обычно показывает хорошую скорость сходимости для широкого класса задач. По сравнению с *GMRES, BiCGStab* экономичнее по памяти и часто быстрее, но менее стабилен.

В рамках курсовой работы в качестве основного численного метода выбран метод Гаусса [3]. Это решение обусловлено простотой реализации, полным отсутствием настраиваемых параметров, минимальными требованиями к дополнительной памяти, прозрачными зависимостями по данным (что удобно для параллелизации) и возможностью сравнения последовательной, многопоточной и распределённой версий без необходимости введения сложных механизмов предобусловливания и критериев остановки.

1.2 Выбор архитектуры распределенной системы

1.2.1. Децентрализованная архитектура является одной из наиболее распространённых вариантов, часто используется в высокопроизводительных библиотеках на базе *MPI* [4]. В ней все узлы равноправны, нет выделенного координатора: каждый процесс владеет своей частью строк матрицы. На каждом шаге

исключения процессы самостоятельно определяют, кто владеет ведущей строкой – для этого выполняется глобальное сравнение. После выбора ведущей строки она рассылается всем процессам через коллективную операцию *Broadcast*. Затем каждый процесс локально обновляет свою часть матрицы, вычитая кратные копии ведущей строки. Синхронизация шагов обеспечивается глобальными барьерами или неявно через коллективные операции. Схема позволяет добиться высокой масштабируемости, минимизирует роль центрального узла и снижает риск единой точки отказа. Однако она требует тщательной реализации коллективных примитивов и очень чувствительна к ошибкам в логике обмена данными. Схема децентрализованной архитектуры представлена на рисунке 1.1.

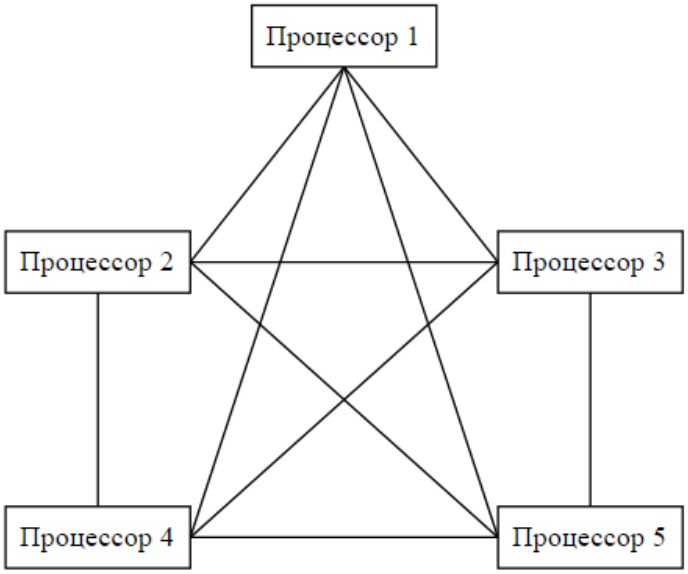


Рисунок 1.1 – Схема децентрализованной архитектуры

В децентрализованной архитектуре ключевым фактором масштабируемости является использование коллективных операций, которые позволяют избежать явного указания отправителя и получателя. Основная проблема – рост числа коммуникаций с увеличением числа процессов**.**

1.2.2. Другой популярный подход – двумерное блочное циклическое распределение Матрица разбивается на квадратные блоки фиксированного размера $b \times b$, которые распределяются по двумерной сетке процессов ($m \times n$).

Распределение циклическое: блок (i, j) попадает к процессу с координатами ($i \bmod m$, $j \bmod n$), т.е. по остатку от деления. На каждом шаге ведущий блок (панель) передаётся по строкам и столбцам сетки процессов: сначала выполняется поиск ведущего элемента внутри колонки процессов, владеющей текущим столбцом, затем осуществляется рассылка по строке панели по строке процессов, а обновление нижележащих блоков происходит локально с использованием операций *BLAS*. Схема показана на рисунке 1.2.

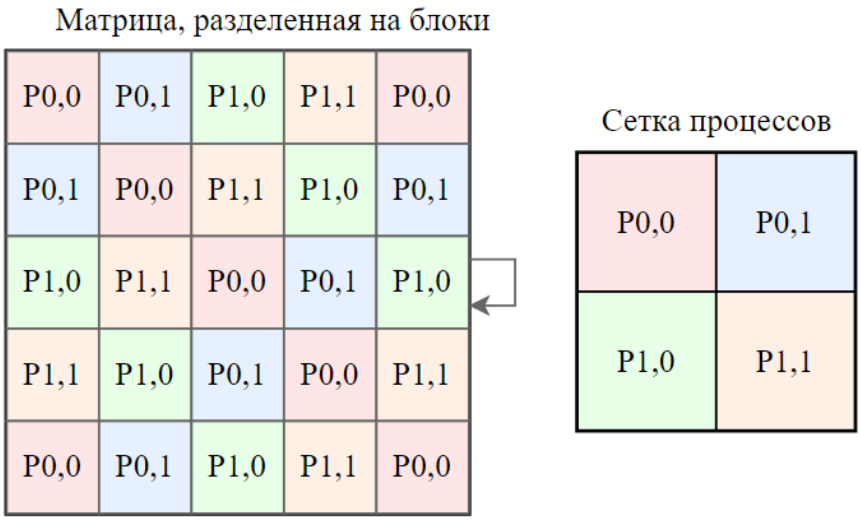


Рисунок 1.2 – Схема двумерного блочного циклического распределения

Двумерное блочное распределение эффективно на поздних этапах алгоритма, когда большая часть матрицы уже исключена: коммуникации ограничиваются только активными блоками в нижнем правом углу, а локальные операции *BLAS* максимально используют кэш-память и векторные инструкции процессора. Подбор размера блока *b* играет ключевую роль: слишком маленький размер приводит к большому количеству мелких сообщений и накладным расходам на коммуникации, слишком большой – к дисбалансу нагрузки между процессами и ухудшению локальности доступа к данным. Размер блока обычно выбирают в диапазоне 64-256 элементов в зависимости от характеристик процессора, скорости сети и объёма задачи.

1.2.3. В архитектуре «*Master-Worker*» («координатор-рабочие узлы») выделяется один центральный процесс – *Master* (координатор), который берёт на себя управляющую роль, а остальные процессы – *Worker*'ы (рабочие узлы) выполняют только локальные вычисления по командам координатора. Координатор загружает исходные данные, разбивает матрицу на строки и распределяет их по узлам. На каждом шаге прямого хода алгоритма Гаусса координатор определяет, какой узел владеет текущей строкой, запрашивает у него ведущую строку, находит в ней максимальный по модулю элемент (ведущий элемент), рассылает выбранную ведущую строку всем рабочим узлам, после чего каждый узел независимо обновляет свою часть матрицы и правой части, вычитая кратные копии ведущей строки. После завершения обновления узлы отправляют координатору подтверждение о готовности к следующему шагу. По окончании прямого хода координатор собирает полученную матрицу и модифицированный вектор свободных членов, после чего может выполнить обратный ход либо самостоятельно, либо распределив его аналогичным образом по узлам. Схема архитектуры «*Master-Worker*» представлена на рисунке 1.3.

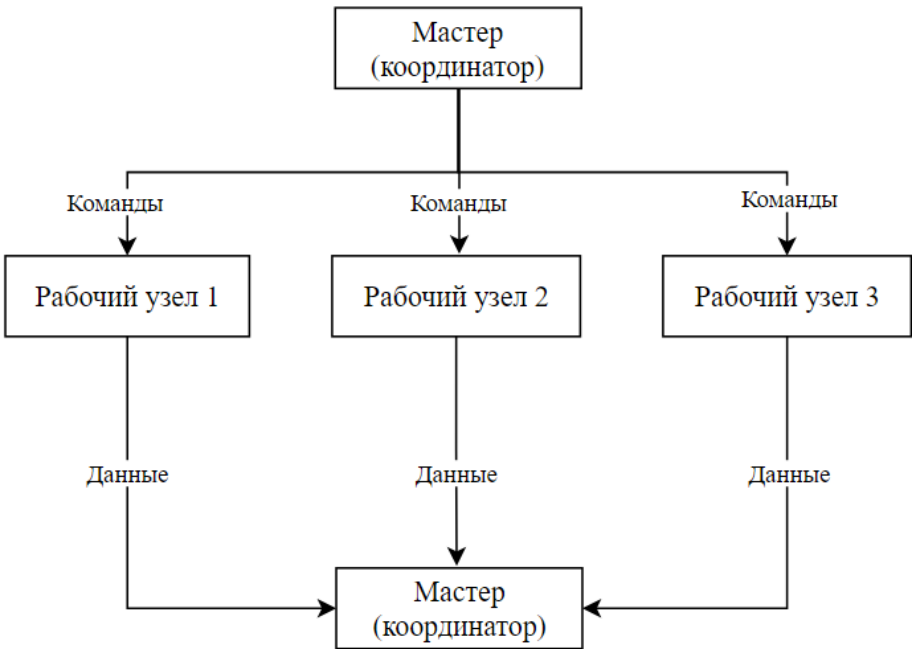


Рисунок 1.3 – Схема архитектуры «*Master-Worker*»

В архитектуре «*Master-Worker*» вся управляющая логика сосредоточена на координаторе. Он отвечает за поиск ведущего элемента, принятие решений о перестановках строк и последовательный контроль шагов алгоритма, что снижает вероятность рассинхронизации процессов.

В настоящей работе выбрана архитектура «*Master-Worker*» («координатор -рабочие узлы») [6]. Выбор обусловлен тем, что она проще в реализации и отладке по сравнению с децентрализованными или двумерными схемами: вся логика принятия решений сосредоточена в одном узле, отсутствует необходимость в сложных коллективных операциях и глобальной синхронизации.

1.3 Выбор средств реализации распределенной системы

1.3.1. В качестве языка программирования для реализации распределённой системы решения СЛАУ рассматривались несколько вариантов.

C++ даёт прямой низкоуровневый доступ к памяти, позволяет полностью использовать современные векторные инструкции процессора (*AVX2*, *AVX*-512), обеспечивает минимальные накладные расходы на выполнение кода и поддерживает библиотеки параллелизма (*OpenMP*, *TBB*, *MPI*) [7].

Преимущества:

– контроль над использованием кэш-памяти и локальностью данных;

– огромная экосистема библиотек для линейной алгебры (*Eigen*, *Blaze*);

– поддержка современных *SIMD*-инструкций.

Недостатки:

– необходимость вручную управлять памятью;

– сложность написания и отладки многопоточного кода и сетевого взаимодействия;

Python с библиотеками *NumPy*, *SciPy* и *mpi4py* предоставляет средства работы с многомерными массивами, встроенную поддержку распределённых вычислений и средства для визуализации, анализа данных [8].

Преимущества:

– простота отладки, интерактивной работы и визуализации результатов;

– встроенные средства для массивов и матриц (*NumPy*, *SciPy.sparse*);

– встроенная поддержка распределённых вычислений через *mpi4py*.

Недостатки:

– низкая производительность из-за интерпретируемости и *GIL*;

– отсутствие прямой векторизации;

– не подходит для больших задач, сравнения производительности;

Java предоставляет автоматическое управление памятью, встроенные средства многопоточности, возможности асинхронного кода и полную кроссплатформенность [9].

Преимущества:

– стабильная платформа с автоматическим управлением памятью;

– встроенная поддержка многопоточности и пулов потоков;

– средства асинхронного программирования;

– обработка сетевых событий;

– кроссплатформенность.

Недостатки:

– экспериментальный и нестабильный *Vector API*;

– *JIT*-компилятор уступает современному *.NET;*

C# предоставляет встроенные средства многопоточности (*Parallel.For*, *Task Parallel Library*, *PLINQ*, *async*/*await*), классы для сетевого взаимодействия (*TcpClient*, *TcpListener*, *NetworkStream*) и прямую поддержку векторных инструкций процессора (*System.Runtime.Intrinsics* с *AVX2*/*AVX*-512) [10].

Преимущества:

– встроенные средства параллельного и асинхронного кода;

– классы для сетевого взаимодействия;

– классы для бинарной сериализации данных;

– прямая поддержка векторных инструкций процессора;

– автоматическое управление памятью.

Недостатки:

– сборка мусора (*GC*) может вызывать короткие паузы;

– выше накладные расходы на запуск по сравнению с *C*++;

– зависимость от версии *.NET*.

Выбор *C#* в качестве языка программирования обусловлен сочетанием высокой производительности, удобства и скорости разработки, наличию автоматическому управлению памятью, встроенных средств для параллельного и сетевого программирования, а также минимального количества внешних зависимостей.

1.3.2. При выборе интегрированной среды разработки (*IDE*) рассматривались три основных варианта: *Visual Studio* 2022, *JetBrains Rider* и *IntelliJ IDEA Community Edition* (с плагином для *.NET*).

JetBrains Rider – IDE, разработанная специально для *.NET*-разработки.

Преимущества:

– удобные инструменты рефакторинга;

– встроенная поддержка *NuGet*, *Git*, *unit*-тестов;

– *IntelliSense*, автодополнение и поддержка всех возможностей *.NET*;

– кроссплатформенность.

Недостатки:

– производительность заметно ниже, чем у *Visual Studio*;

– отладчик уступает встроенному в *Visual Studio*;

– требует отдельной лицензии;

– больший расход ресурсов на запуск и работу.

IntelliJ IDEA Community Edition (с плагинами для *.NET*) – базовая версия *IDE* от *JetBrains*, которая может использоваться для *C#*-проектов.

Преимущества:

– не требует отдельной лицензии;

– кроссплатформенность;

– имеются инструменты рефакторинга, навигации и поиска по проекту;

– интеграция с *Git*.

Недостатки:

– требует установки дополнительных плагинов (*ReSharper* или *.NET Tools*);

– инструменты диагностики асинхронных операций отсутствуют;

– настройка окружения требует много времени.

Visual Studio 2022 (*Community Edition*) –интегрированная среда разработки от *Microsoft*, которая остаётся эталоном для проектов на *.NET*.

Преимущества:

– отладчик с поддержкой многопоточности;

– поддержка сетевой отладки;

– встроенный профилировщик производительности;

– возможность одновременного запуска и отладки нескольких процессов;

– *IntelliSense*, автоматический анализ кода (*Roslyn analyzers*), интеграция с *Git*, *NuGet*, *Live Share*;

– полная поддержка всех новых возможностей *.NET*.

Недостатки:

– более высокое потребление ресурсов по сравнению с *JetBrains Rider*;

– основная функциональность привязана к *Windows*.

Выбор *Visual Studio* 2022 в качестве основной среды разработки обусловлен её возможностями отладки и профилирования многопоточных и распределённых приложений.

2 АЛГОРИТМИЧЕСКИЙ АНАЛИЗ ЗАДАЧИ

2.1 Архитектурные особенности реализации методов решения СЛАУ

В основе разрабатываемого программного обеспечения лежит концепция разделения ролей между центральным координатором и сетью вычислительных узлов (воркеров). Координатор берет на себя функции управления жизненным циклом задачи: загрузку исходных данных, их предварительную обработку, распределение нагрузки между узлами и сбор финальных результатов. Вычислительные узлы функционируют как изолированные процессы, принимающие сегменты данных и выполняющие над ними элементарные математические операции в рамках метода исключения Гаусса.

Сетевое взаимодействие между компонентами системы реализовано через стандартные сокеты по протоколу *TCP*/*IP*, что гарантирует сохранность и правильную последовательность передаваемых чисел. Весь обмен данными происходит в бинарном виде, что исключает лишние преобразования и позволяет использовать всю доступную ширину сетевого канала для передачи полезной информации. Каждый воркер хранит в оперативной памяти только ту часть матрицы, за которую он несет ответственность, что важно при обработке систем сверхбольшой размерности, превышающих объем памяти одного вычислительного узла.

Алгоритм решения предусматривает два режима функционирования: последовательный и параллельный. В последовательном режиме все вычисления производятся в рамках одного потока. Параллелизм в приложении реализован на двух разных уровнях. Первый уровень – это распределение работы между разными узлами в сети. Второй уровень находится внутри каждого отдельного узла и использует возможности процессоров по векторной обработке.

Для преодоления ограничений оперативной памяти при работе с матрицами размером в десятки тысяч уравнений, в архитектуру заложена технология работы через проекцию файлов. Суть её в том, что огромная матрица не загружается в память целиком, а «отображается» на жесткий диск. Операционная система сама решает, какие части файла нужно подгрузить в память в конкретный момент, а какие – выгрузить обратно на диск. Для программы это выглядит как работа с огромным массивом в памяти, но на самом деле данные физически хранятся в файле.

Сам вычислительный процесс организован как чередование фаз обмена и расчета. На каждом шаге прямого хода метода Гаусса координатор определяет, у какого воркера находится текущая «главная» (ведущая) строка. Этот воркер считывает её и передает координатору, а тот рассылает её всем остальным участникам. Получив ведущую строку, все узлы одновременно начинают модифицировать свои локальные данные. После того как матрица приведена к треугольному виду, начинается обратный ход, в ходе которого узлы передают вычисленные значения неизвестных обратно координатору для сборки финального ответа и проверки точности. Такая структура позволяет легко масштабировать систему.

Функциональная схема архитектуры приложения для решения СЛАУ методом Гаусса приведена на рисунке 2.1.

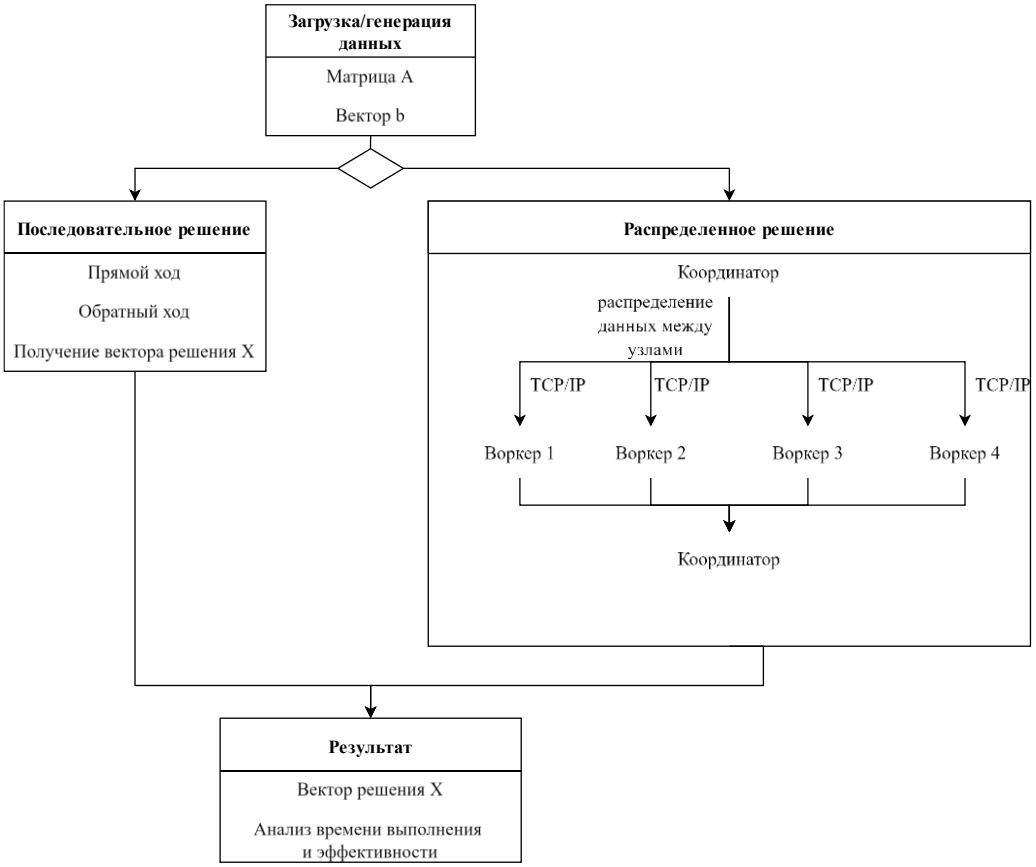


Рисунок 2.1 – Функциональная схема архитектуры приложения

Полученная архитектура обеспечивает баланс между производительностью и скоростью распределенной системы и точностью получаемых результатов.

2.2 Последовательный алгоритм решения СЛАУ

Последовательный режим решения является базовым вычислительным процессом, в котором все этапы алгоритма Гаусса выполняются строго детерминировано в рамках одного программного потока. В отличие от распределенного режима, здесь отсутствуют сетевые задержки и расходы на синхронизацию воркеров, однако критическое значение приобретает эффективность управления локальными ресурсами процессора и оперативной памяти.

Процесс начинается с этапа инициализации, где вычислительное ядро получает доступ к структурам данных, подготовленным на предыдущих стадиях.

Основной цикл алгоритма состоит из прямого хода – итерационного приведения матрицы к верхнетреугольному виду. На каждой итерации этого этапа система выполняет поиск опорного элемента в текущем столбце. Для повышения численной устойчивости реализован алгоритм частичного выбора ведущего

элемента по столбцу: строки переставляются таким образом, чтобы в качестве делителя выступало число с максимальным абсолютным значением. Данная процедура важна для работы с плохо обусловленными матрицами, так как она минимизирует накопление погрешностей округления при выполнении операций деления на малые величины.

Алгоритм задействует аппаратное ускорение на уровне команд процессора через использование векторных расширений. Это позволяет за одну итерацию внутреннего цикла обрабатывать пачку из четырех или восьми коэффициентов уравнения. Прямой доступ к памяти позволяет процессору не тратить такты на проверку границ массивов и другие сервисные операции среды исполнения.

После того как матрица приведена к треугольному виду, система переходит в состояние обратного хода. Вычисленные значения неизвестных помещаются в выходной буфер. Завершается работа последовательного режима переходом в состояние верификации, где выполняется расчет вектора невязки $||Ax - b||$.

На рисунке 2.2 приведена диаграмма состояний последовательного режима решения СЛАУ методом Гаусса.

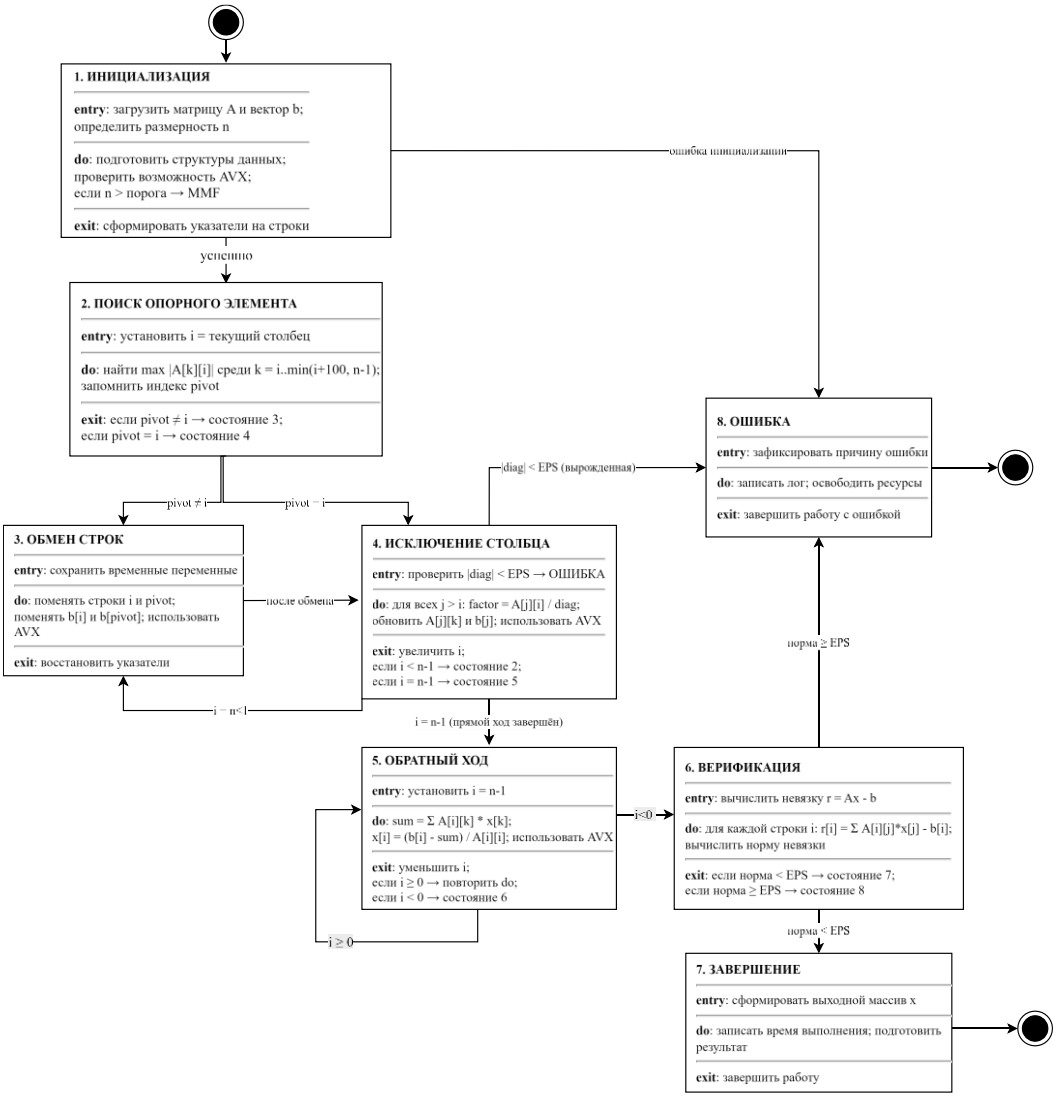


Рисунок 2.2 – Диаграмма состояний последовательного алгоритма

Как видно из рисунка 2.2, диаграмма начинается с состояния 1 «Инициализация», где выполняются загрузка матрицы и вектора, подготовка структур данных и формирование указателей на строки. Далее следует переход к состоянию 2 «Поиск опорного элемента», в котором находится максимальный по модулю элемент в текущем столбце, после чего по условию «$pivot \neq i$» осуществляется переход к состоянию 3 «Обмен строк» с последующим переходом в состояние 4, либо по условию «$pivot = i$» прямой переход к состоянию 4 «Исключение столбца», где происходит исключение переменных и проверка диагонального элемента на вырожденность. Из состояния 4 возможны три варианта: возврат по циклу к состоянию 2 для следующего столбца (при $i < n$-1), переход при завершении прямого хода ($i = n$-1) к состоянию 5 «Обратный ход», либо аварийный переход к состоянию 8 «Ошибка» при вырожденной матрице. Состояние 5 выполняет обратную подстановку с итерационным уменьшением i, при $i \geq 0$ повторяя вычисления, а при $i < 0$ переходя к состоянию 6 «Верификация», где рассчитывается норма невязки, после чего в зависимости от её значения происходит переход либо к состоянию 7 «Завершение» (при норме меньше eps), либо к состоянию 8 «Ошибка» (при норме больше либо равно eps). Кроме того, предусмотрен аварийный переход из состояния 1 в состояние 8 при ошибке инициализации.

2.3 Распределенный алгоритм решения СЛАУ

Распределенный алгоритм решения базируется на архитектуре «мастер-воркер», где координатор управляет распределением вычислительной нагрузки между множеством независимых вычислительных узлов. Реализация данного алгоритма ориентирована на решение задач большой размерности, не помещающихся в оперативную память одного вычислительного узла.

Процесс начинается с этапа инициализации, в ходе которого координатор запускает необходимое количество воркеров, определяемое как минимум из числа процессорных ядер и фиксированного лимита. Каждому воркеру передаются сетевые параметры, после чего координатор ожидает установления соединений. Далее координатор распределяет исходные данные: матрица системы и вектор свободных членов разбиваются по строкам между воркерами, что обеспечивает равномерную загрузку всех вычислительных узлов. Каждый воркер получает свой набор строк, которые сохраняются в локальной памяти процесса, а также соответствующие элементы вектора правой части. Тогда общее количество строк, закрепленных за каждым воркером, определяется по формуле 2.1:

$$rows = \left\lceil \frac{n-r}{p} \right\rceil \tag{2.1}$$

Для 4 воркеров с рангами 0, 1, 2 и 3 кол-во строк будет по 250 строк на каждого воркера соответственно, пример расчета показан в формуле 2.2 (для воркера 1 с рангом 0):

$$rows_1 = \left\lceil \frac{1000-0}{4} \right\rceil = 250 \qquad (2.2)$$

Распределение выполняется по принципу кругового обхода *(round-robin)*, что позволяет равномерно загрузить все узлы даже при неполном делении. Для воркера с рангом r (r=0,1,2,3) множество индексов строк определяется по формуле 2.3:

$$I_r = \{r, r+p, r+2p, ...\} \bigcap [0, n-1] \qquad (2.3)$$

в результате чего воркер 1 с рангом 0 получает строки 0, 4, 8, …, 996 (250 строк), воркер 2 с рангом 1 получает строки 1, 5, 9, …, 997 (250 строк), воркер 3 с рангом 2 получает строки 2, 6, 10, …, 998 (250 строк), воркер 4 с рангом 3 получает строки 3, 7, 11, …, 999 (250 строк). Такой подход гарантирует, что строки с близкими номерами не попадают в один узел, что снижает вероятность дисбаланса при частичном выполнении прямого хода.

На рисунке 2.3 приведена схема распределения строк между воркерами для данного примера.

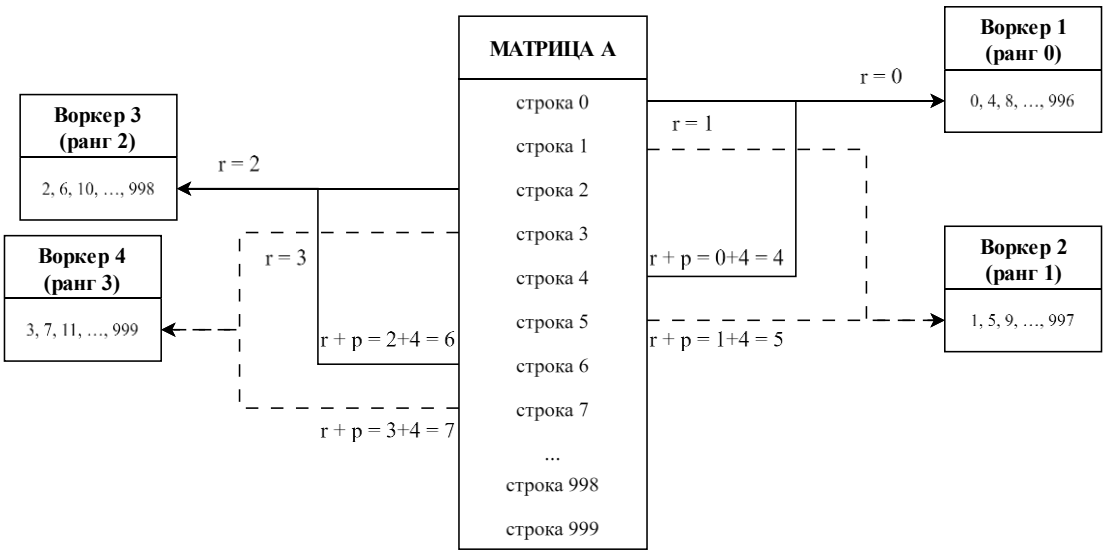


Рисунок 2.3 – Схема распределения строк между узлами

Основной цикл алгоритма представляет собой распределенный прямой ход метода Гаусса. На каждой итерации координатор определяет воркера, владеющего текущей ведущей строкой, и запрашивает у него ведущий элемент матрицы и соответствующее значение вектора правой части. Получив от воркера эти данные, координатор рассылает ведущую строку, ведущий (диагональный) элемент и значение правой части всем остальным воркерам.

Каждый воркер, получив ведущие данные, выполняет локальное исключение в закрепленных за ним строках (рисунок 2.4): для каждой строки, чей индекс больше текущего, вычисляется коэффициент (отношение элемента текущего

столбца к ведущему элементу), и если его абсолютное значение превышает заданный порог, производится обновление коэффициентов строки и соответствующего элемента вектора правой части с использованием векторных инструкций.

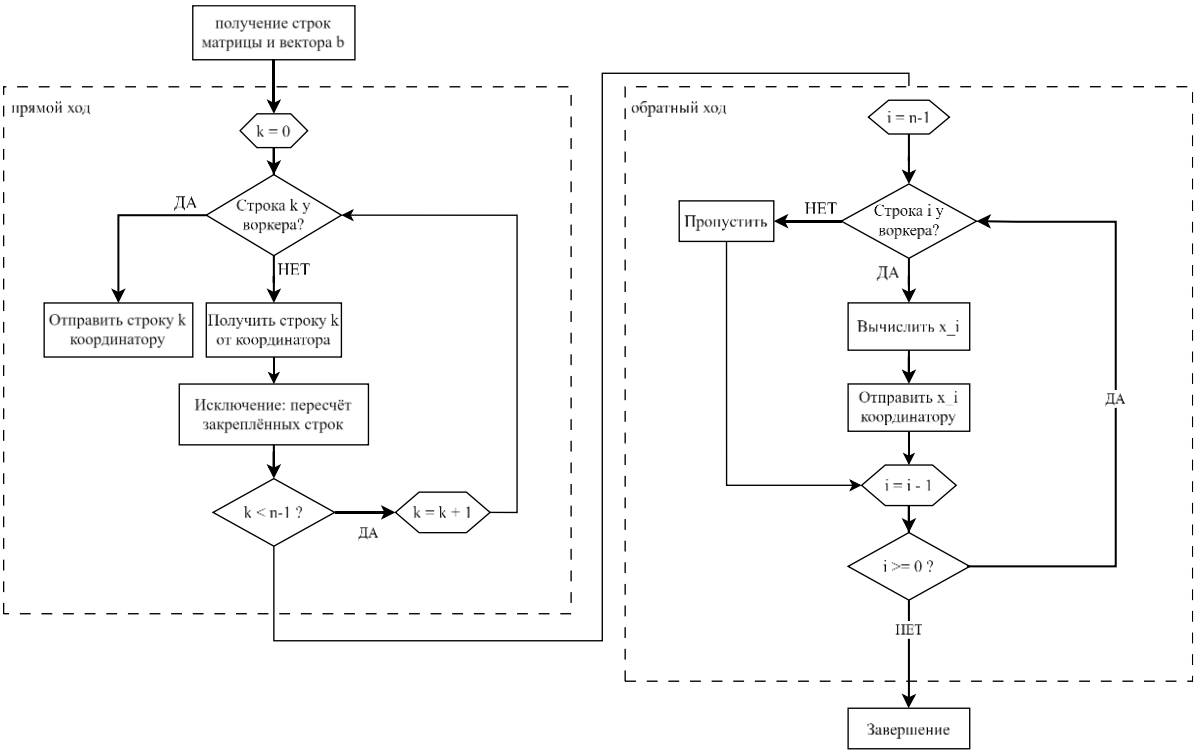


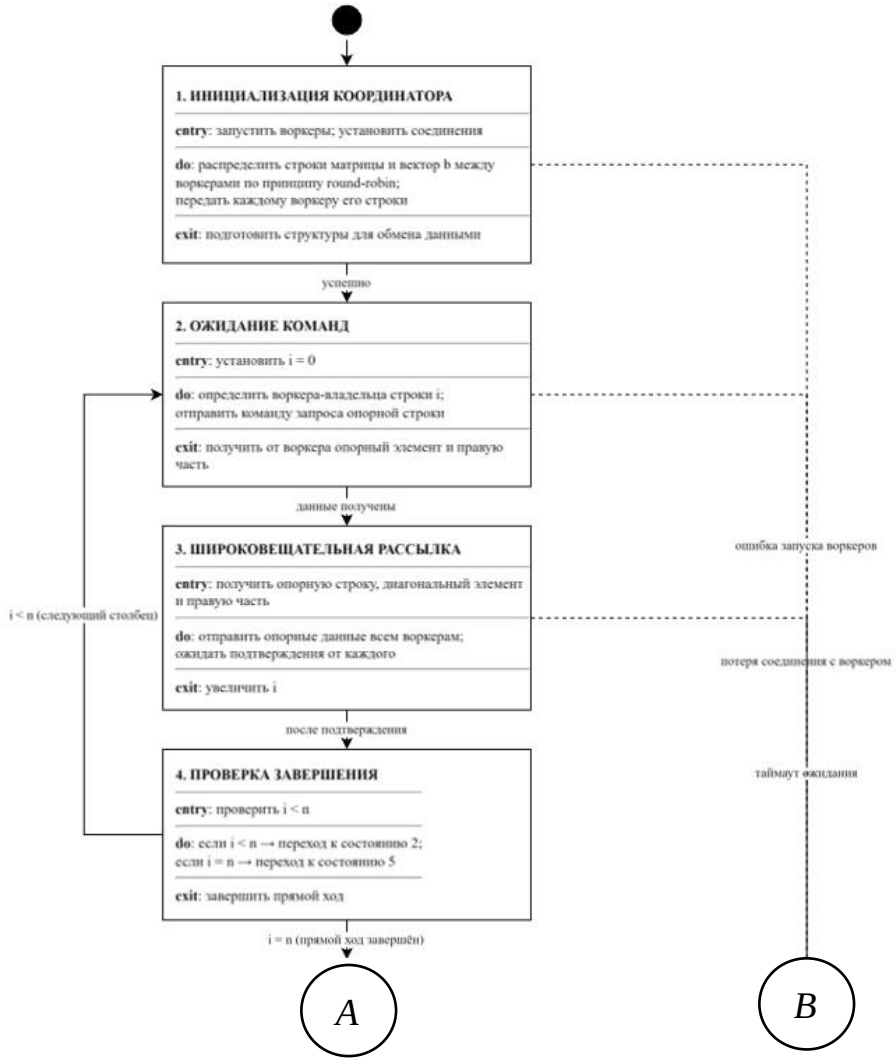
Рисунок 2.4 – Схема работы конкретного воркера

При прямом ходе после инициализации счётчика $k = 0$ воркер проверяет, принадлежит ли ему строка с индексом k. Если строка принадлежит воркеру, он отправляет её координатору для последующей широковещательной рассылки другим узлам, после чего сразу переходит к локальному исключению, используя собственную копию строки. Если строка не принадлежит воркеру, он ожидает получения этой строки от координатора и только затем выполняет исключение. Операция исключения заключается в пересчёте всех закреплённых за воркером строк с индексами, большими k: каждая такая строка заменяется разностью между собой и ведущей строкой, умноженной на коэффициент. После завершения исключения проверяется условие $k < n$-1: если оно истинно, счётчик k увеличивается на единицу, и цикл повторяется для следующего столбца; если ложно – прямой ход завершается, и управление переходит к обратному ходу.

При обратном ходе воркер инициализирует счётчик $i = n$-1 (начинает с последней строки). Для каждого i проверяется принадлежность строки воркеру. Если строка принадлежит, воркер вычисляет значение неизвестной x_i по формуле, используя уже найденные значения x_j для $j > i$, и отправляет результат координатору. Если строка не принадлежит, воркер пропускает эту итерацию. Затем счётчик i уменьшается на единицу, и проверяется условие $i \geq 0$: при истинности цикл повторяется для следующей строки; при ложности работа воркера завершается.

После завершения всех итераций прямого хода координатор отправляет воркерам команду на завершение и собирает от каждого из них вычисленные значения неизвестных, соответствующие закрепленным строкам. Сборка итогового вектора решения осуществляется на стороне координатора. Команда завершения передается в виде пакета, который воркеры распознают после обработки всех полученных опорных строк. Это гарантирует, что ни один воркер не завершится преждевременно, пока хотя бы один узел продолжает выполнять операции исключения.

Процесс сбора результатов организован таким образом, что каждый воркер после завершения прямого хода и подстановки передает координатору количество обработанных строк, а затем для каждой строки – её индекс и вычисленное значение неизвестной. Координатор, принимая эти данные, последовательно заполняет итоговый массив решения, восстанавливая исходный порядок неизвестных. На рисунке 2.5 приведена диаграмма состояний параллельного алгоритма решения СЛАУ методом Гаусса, реализующая архитектуру мастер-воркер.



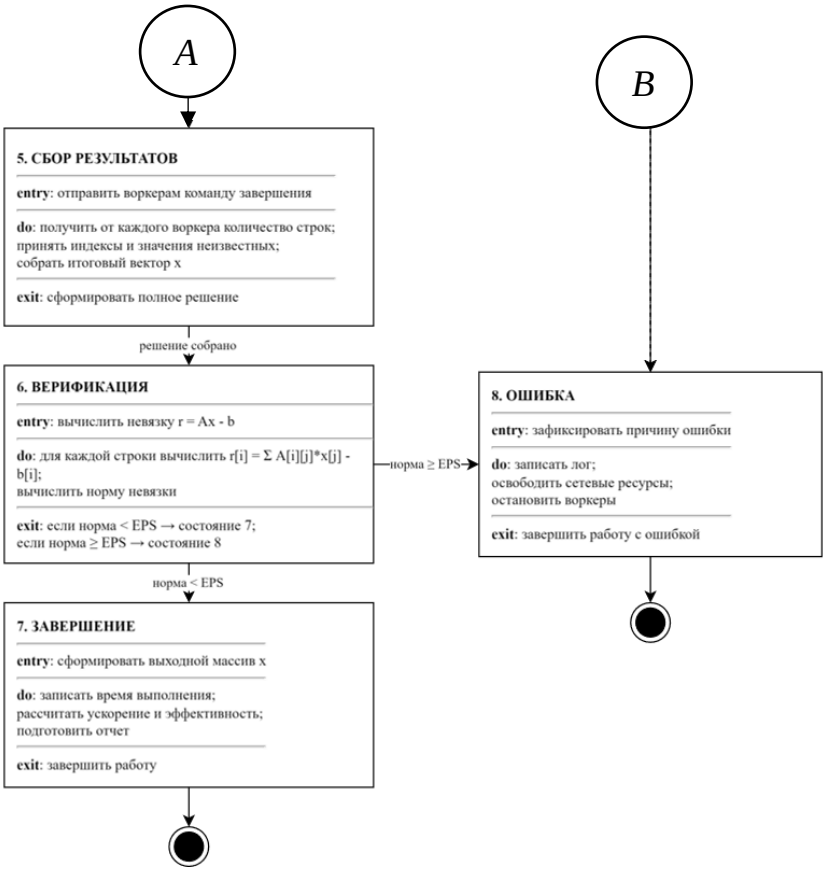


Рисунок 2.5 – Диаграмма состояний параллельного алгоритма

Параллельный алгоритм начинается с состояния 1 «Инициализация координатора», где выполняются запуск воркеров, установление *TCP*-соединений, распределение строк матрицы и вектора правой части между воркерами, после чего осуществляется переход к состоянию 2 «Ожидание команд», в котором координатор определяет воркера-владельца текущей опорной строки и отправляет ему запрос на получение опорного элемента и соответствующего значения правой части; при успешном получении данных происходит переход к состоянию 3 «Широковещательная рассылка», где координатор отправляет опорную строку, диагональный элемент и значение правой части всем воркерам и ожидает от каждого подтверждения выполнения локального исключения, после чего переходит к состоянию 4 «Проверка завершения»: если номер текущего столбца меньше размерности системы, осуществляется возврат к состоянию 2 для обработки следующего столбца, а если все столбцы обработаны, выполняется переход к состоянию 5 «Сбор результатов», где координатор отправляет воркерам команду завершения, принимает от каждого вычисленные значения неизвестных с соответствующими индексами и формирует итоговый вектор решения; далее следует переход к состоянию 6 «Верификация», где рассчитывается норма невязки, после чего при значении меньше машинного допуска осуществляется переход к состоянию 7 «Завершение» с формированием отчета об ускорении и эффективности, а при превышении допуска – к состоянию 8 «Ошибка»; также предусмотрены аварийные переходы, обозначенные пунктирными линиями, из состояний 1, 2 и 3 в состояние 8 при ошибках инициализации, потере соединения с воркером или превышении таймаута ожидания, а завершающие состояния обозначены черными кругами для успешного и ошибочного завершений соответственно.

3 РАЗРАБОТКА, ТЕСТИРОВАНИЕ И ВЕРИФИКАЦИЯ РАСПРЕДЕЛЕННОЙ СИСТЕМЫ

3.1 Состав программных модулей и структура классов приложения

Разработанная распределённая система решения СЛАУ методом Гаусса реализована на языке *C#* в рамках платформы *.NET* и представляет собой единое решение, состоящее из нескольких взаимосвязанных проектов. Приложение включает пять основных пространств имён: *Gaussian.Shared, Gaussian.Solver, Gaussian.Worker, Gaussian.Coordinator* и *Gaussian.Client,* что обеспечивает чёткое разделение ответственности, удобство сопровождения и возможность независимой компиляции отдельных компонентов. Листинг расположен в приложении А.

Проект *Gaussian.Shared* содержит общие модели данных, используемые всеми модулями. Центральным классом здесь является *SparseMatrix* – разреженное представление матрицы коэффициентов в виде массива списков *MatrixElement*, где каждый элемент хранит индекс столбца и значение. Также определены модели *SolveRequest, SolveResponse, SolveTask, SessionStats* и *WorkerBenchmark* для передачи параметров задач, результатов расчётов и статистических данных.

Вычислительное ядро сосредоточено в проекте *Gaussian.Solver* в классе *GaussEngine*. Класс реализует методы *SolveSequential* и *SolveParallel* для последовательного и параллельного решения систем.

Gaussian.Worker представляет собой консольное приложение, запускаемое как отдельный процесс рабочего узла. В классе *Worker* метод *Main* обрабатывает параметры командной строки (размерность системы, ранг узла, общее количество узлов и время выполнения), вычисляет закреплённый за узлом диапазон строк.

Координатор системы реализован в проекте *Gaussian.Coordinator* классом *SolutionCoordinator*. Он управляет очередью задач, запускает необходимое количество процессов *Worker.exe*, ожидает их завершения. Вспомогательные классы *DataManager* обеспечивают загрузку матрицы и вектора свободных членов из текстовых файлов, а *DataGenerator* – генерацию тестовых систем.

Клиентская часть приложения находится в *WPF*-проекте *Gaussian.Client*. Графический интерфейс позволяет пользователю выбирать файлы матрицы и вектора, загружать пакет тестовых систем, генерировать новые тестовые матрицы, запускать расчёты с заданным количеством рабочих узлов (2, 4, 6 или 8), просматривать подробный лог выполнения и анализировать результаты в виде интерактивных диаграмм (сравнение времени, загрузка узлов, зависимость производительности от количества узлов).

3.2 Реализация вычислительного ядра и алгоритмов решения

3.2.1 Последовательное решение реализовано в классе *GaussEngine* проекта *Gaussian.Solver*. Метод *SolveSequential* вызывает внутренний метод *InternalSolve*, который выполняет полный алгоритм Гаусса с частичным выбором главного элемента.

Для плотных матриц коэффициенты размещаются в файле *gauss_cache.bin* через механизм *MemoryMappedFile*. Это позволяет работать с матрицами очень большой размерности, так как данные хранятся на диске, а доступ к ним осуществляется как к обычному массиву в памяти. После отображения файла матрица обрабатывается как непрерывный блок памяти через указатель *double* a*.

На этапе прямого хода для каждого столбца *i* производится поиск максимального по модулю элемента в текущем столбце ниже диагонали. Найденная строка при необходимости переставляется с *i*-й строкой с помощью метода *SwapRows*. После этого для всех строк $j > i$ вызывается метод *EliminateFast*, который вычисляет фактор деления и вычитает кратную ведущую строку из строки *j*. Оба метода (*SwapRows* и *EliminateFast*) используют векторные инструкции *AVX* через *Vector256<double>*, что позволяет обрабатывать по четыре элемента матрицы за одну операцию.

Обратный ход реализован с нисходящим циклом по индексу *i* от *n*-1 до 0. Для ускорения вычисления суммы элементов выше диагонали также применяется *AVX* (*Avx.Multiply* и *Avx.Add*). В результате формируется вектор решения *x*. Для ленточных (разреженных с ограниченной шириной ленты) матриц используется специализированный метод *SolveBanded*. В нём матрица хранится в компактном двухмерном массиве *band*[][], содержащем только главную диагональ и прилегающие к ней элементы в пределах вычисленной ширины ленты. Это существенно снижает расход памяти и ускоряет вычисления за счёт исключения операций с нулевыми элементами.

3.2.2 Распределённое решение СЛАУ реализовано на основе архитектуры *Master-Worker* с использованием проекта *Gaussian.Worker*. Каждый рабочий узел представляет собой независимый процесс, запускаемый координатором при старте вычислений. Координатор формирует команду запуска для каждого узла, передавая через аргументы командной строки следующие параметры: размерность системы *n*, ранг узла *rank* (целое число от 0 до *total* - 1), общее количество узлов *total*.

При запуске процесса в *Main* воркер разбирает аргументы и по принципу *round-robin* определяет закреплённый за ним диапазон строк матрицы. Затем создаётся экземпляр *GaussEngine*. Через *typeof*(*GaussEngine*).*GetMethod* получается ссылка на приватный метод (в дальнейшем вызывается многократно). В вычислительном цикле на каждой итерации определяется индекс строки из закреплённого диапазона. Если индекс выходит за границы матрицы, он возвращается к началу диапазона. Через *MethodInfo* вызывается метод *EliminateFast* с параметрами: указатель на данные матрицы в отображённом файле, вектор свободных

членов, индекс строки, начальный индекс, ограничение по столбцам и диагональный элемент. Метод для всех нижележащих строк зоны ответственности узла вычисляет фактор деления и вычитает ведущую строку, умноженную на этот фактор. Для ускорения используются *AVX*-инструкции.

После каждой второй итерации в консоль выводится: ранг узла, его диапазон строк, текущий индекс и слово *ACTIVE*. По завершении прямого хода начинается фаза синхронизации – с паузами и выводом *SYNC*: *OK*. Метод *Main* возвращает 0, процесс завершается.

3.3 Реализация пользовательского интерфейса

Пользовательский интерфейс приложения реализован в виде *WPF*-приложение и находится в проекте *Gaussian.Client*. Экранные копии расположены в приложении Б. Разметка интерфейса определена в файле *MainWindow.xaml* и использует тёмную цветовую схему.

Первая вкладка «Главная» (рисунок Б.1 в приложении Б) разделена на две колонки. В левой колонке размещены элементы управления загрузкой данных: кнопка «Выбрать матрицу», кнопка «Выбрать вектор *b*» и кнопка «Выбрать папку (Пакет)», позволяющие выбрать файл матрицы, файл вектора свободных членов или папку с пакетом задач соответственно. Кнопка «Запустить расчет» запускает процесс решения. Ход выполнения вычислений и результаты выводятся в поле лога, расположенном в правой части вкладки.

Вторая вкладка «Аналитика» (рисунок Б.2 в приложении Б) предназначена для отображения агрегированных статистических данных по всем выполненным за текущую сессию задачам. В верхней части вкладки расположены шесть информационных карточек. Карточка «Ср. послед.» отображает среднее время последовательного выполнения систем. Карточка «Ср. параллел.» показывает среднее время параллельного выполнения и связана с полем. Карточка «Ср. ускорение» демонстрирует усреднённый коэффициент ускорения, достигнутый при распараллеливании. Карточка «Время обслуж.» содержит среднее время обслуживания одной задачи. Карточки «λ (вход)» и «μ (выход)» показывают интенсивность входного потока задач и интенсивность обслуживания соответственно. Ниже расположен блок «Средняя загрузка узлов», который отображает усреднённый по всем решённым задачам процент загрузки вычислительных узлов. Основное пространство вкладки занимают два графика, построенных с использованием элемента управления *Canvas*. Первый график называется «Сравнение времени (последовательно/параллельно)». Высота столбцов пропорциональна затраченному времени, а под каждым столбцом указывается размерность соответствующей матрицы. В нижней части графика выводится легенда, поясняющая соответствие цветов и размерностей матриц.

Второй график (рисунок Б.3 в приложении Б) называется «Загрузка узлов по задачам». Метод *DrawNodeLoadChart* строит столбчатую диаграмму, на которой для каждой решённой задачи отображается уровень загрузки вычислительных узлов в процентах.

Все вычисления координат графических примитивов выполняются динамически на основе свойств *ActualWidth* и *ActualHeight* соответствующих элементов *Canvas*.

Третья вкладка «Тест узлов» (рисунок Б.4 в приложении Б) предназначена для оценки масштабируемости системы. В верхней части расположен информационный блок и кнопка «Запустить тест всех матриц», запускающая серию расчётов. Тестирование выполняется для всех загруженных матриц с последовательным перебором количества узлов: 1, 2, 4, 6 и 8. По окончанию расчетов пользователю выводится график, отражающий влияние количества узлов на время решения системы.

3.4 Тестирование и верификация

3.4.1 Для проверки работоспособности отдельных компонентов было проведено модульное тестирование. Для организации тестирования был создан отдельный проект *Gaussian.Tests* (листинг в приложении Б), в котором реализованы тесты для всех ключевых модулей с использованием фреймворка *xUnit*.

Класс *SolverTests* проверяет вычислительное ядро системы. Тесты покрывают сценарии решения систем малой размерности (2×2 и 3×3), проверку работы с единичной и диагональной матрицами, обработку разреженных и трёхдиагональных матриц, а также механизм выбора главного элемента при нулевом диагональном элементе. Тесты параллельного режима *SolveParallel* сравнивают результаты с последовательным решением для различного количества потоков (1, 2, 4). Тесты с размерностями 500, 1000 и 2000 проверяют последовательное решение систем, вычисляют невязку полученного решения. Дополнительные тесты проверяют, что параллельное решение с 4 рабочими узлами даёт тот же результат, что и последовательное, а также что решение с 1, 2 и 4 узлами даёт одинаковые результаты.

Класс *DataManagerTests* проверяет функциональность загрузки исходных данных из файлов. Тесты покрывают сценарии чтения матрицы и вектора из текстовых файлов с различными форматами: поддержку пробелов и табуляций в качестве разделителей, корректное считывание десятичных и отрицательных чисел, пропуск пустых строк.

Класс *DataGeneratorTests* проверяет генерацию тестовых систем. Тесты охватывают генерацию случайных систем с заданной размерностью и плотностью заполнения. Дополнительно проверяется, что сгенерированная система является совместной.

Класс *ModelsTests* проверяет корректность работы моделей данных. Тесты покрывают создание и клонирование матрицы *SparseMatrix* с проверкой независимости копии, установку и получение свойств классов *SolveRequest*, *SolveResponse*, *AnalysisResult*, *SolveTask*, *SessionStats* и *WorkerBenchmark*. Проверяются значения по умолчанию, корректность работы с массивами решений и числовыми параметрами, включая время выполнения, ускорение, эффективность, загрузку узлов и норму невязки.

Класс *SolutionCoordinatorTests* проверяет функциональность координатора. Тесты охватывают настройку количества вычислительных узлов и корректное применение этого параметра при решении. Проверяется, что координатор корректно обрабатывает последовательные и параллельные запросы, возвращая согласованные результаты. Тесты покрывают сценарии работы с различным количеством рабочих процессов. Дополнительно тестируется, что возвращаемое решение содержит только конечные числа, а время параллельного выполнения корректно фиксируется в ответе.

На рисунке 3.1 приведены результаты проведения модульного тестирования распределенной системы.

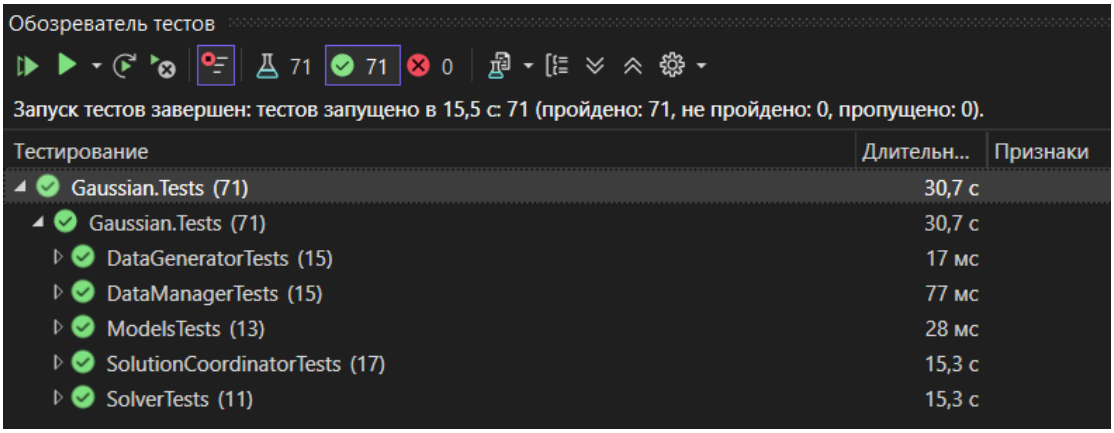


Рисунок 3.1 – Результат выполнения модульного тестирования

Разработанный набор тестов обеспечил высокий уровень покрытия функциональности системы. Система корректно обрабатывает как позитивные сценарии, так и различные исключительные ситуации.

3.4.2 Нагрузочное тестирование распределённой системы проводится для оценки её производительности и масштабируемости в условиях реальных вычислительных задач. Оно позволяет измерить время решения СЛАУ при различной размерности матриц и разном количестве вычислительных узлов, а также количественно оценить эффективность распараллеливания.

Закон Амдала показывает, насколько быстрее выполнится программа фиксированного размера при добавлении дополнительных процессоров. Согласно этому закону, ускорение ограничено последовательной (не распараллеливаемой) частью алгоритма и вычисляется по формуле:

$$S = \frac{1}{f + \frac{1-f}{p}} \tag{3.1}$$

где S – теоретическое ускорение, f – доля последовательных операций (от 0 до 1), p – количество процессоров.

Закон Густафсона-Барсиса показывает, какую задачу большего размера можно решить за то же время, увеличивая количество процессоров. В отличие от закона Амдала, он предполагает масштабирование размера задачи, а не фиксированную вычислительную нагрузку. Формула закона Густафсона-Барсиса имеет вид:

$$S_p = g + (1 - g) \cdot p = (1 - p) \cdot g \tag{3.2}$$

где S_p – масштабированное ускорение, g – доля последовательных операций, p – количество процессоров.

В ходе нагрузочного тестирования разработанной системы были получены результаты, представленные в таблице 3.1.

Таблица 3.1 – Время решения СЛАУ в зависимости от размерности матрицы и количества узлов (в секундах).

Размерность	**1 узел**	**2 узла**	**4 узла**	**6 узлов**	**8 узлов**
500×500	0,08	0,04	0,03	0,02	0,02
1000×1000	0,49	0,31	0,22	0,17	0,17
1500×1500	2,08	1,18	0,95	0,88	0,89
2000×2000	3,98	3,39	2,63	2,47	2,59

На основе полученных данных вычислено ускорение относительно одного узла по формуле:

$$S = \frac{T_1}{T_p} \tag{3.3}$$

где T_1 – время на одном узле, T_p – время на p узлах. Результаты представлены в таблице 3.2.

Таблица 3.2 – Ускорение относительно одного узла

Размерность	**2 узла**	**4 узла**	**6 узлов**	**8 узлов**
500×500	2,00*x*	2,67*x*	4,00*x*	4,00*x*
1000×1000	1,58*x*	2,23*x*	2,88*x*	2,88*x*
1500×1500	1,76*x*	2,19*x*	2,36*x*	2,34*x*
2000×2000	1,17*x*	1,51*x*	1,61*x*	1,54*x*

Для матрицы 500×500 наблюдается хорошее ускорение до 4 узлов (2,67*x*), а дальнейшее увеличение числа узлов даёт незначительный прирост: с 6 до 8 узлов время не меняется (0,02 с). Это объясняется тем, что накладные расходы на коммуникации для матрицы такого размера уже сравнимы с вычислительной нагрузкой на каждом узле. Для матрицы 1000×1000 ускорение растёт до 6 узлов (2,88*x*), после чего выходит на плато (8 узлов дают тот же результат).

Оптимальное количество узлов для данной размерности – 6. Для матрицы 1500×1500 максимальное ускорение достигается на 6 узлах (2,36*x*), а при 8 узлах наблюдается незначительное падение производительности (время увеличивается с 0,88 до 0,89 с). Это свидетельствует о том, что добавление сверх оптимального количества узлов начинает приносить вред из-за возросших затрат на синхронизацию. Для матрицы 2000×2000 ускорение не превышает 1,61*x* даже на 6 узлах, а при 8 узлах время увеличивается (2,59 с вместо 2,47 с). Это объясняется тем, что для матриц большого размера доминирующим фактором становится пропускная способность сети и задержки при передаче ведущих строк между узлами.

На рисунке 3.2 представлен график зависимости времени решения от количества узлов для всех тестируемых размерностей.

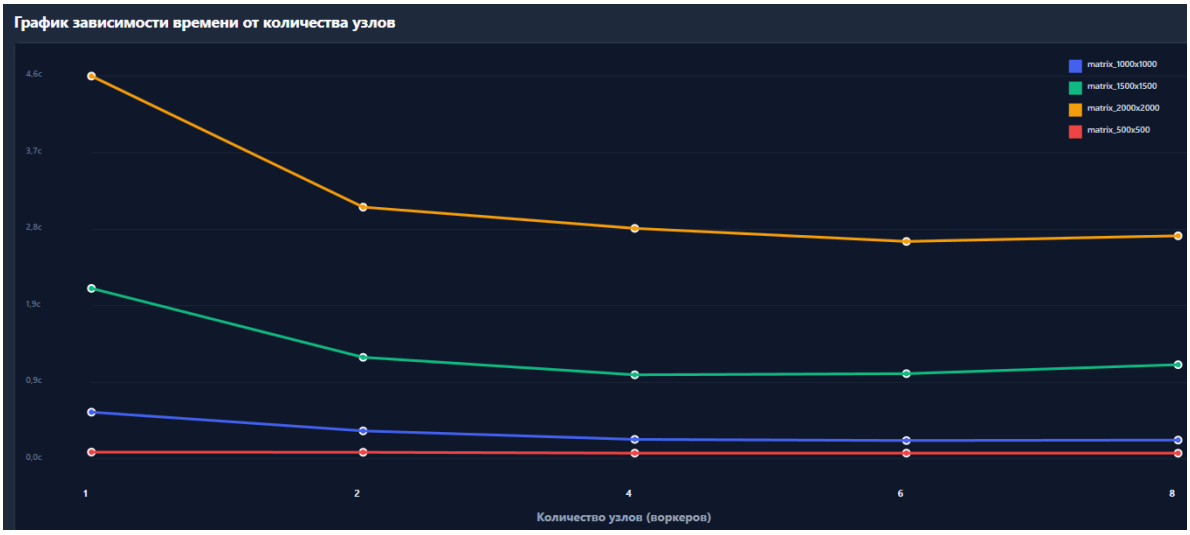


Рисунок 3.2 – График зависимости времени решения от количества узлов

Например, для матрицы размером 1000×1000 было достигнуто ускорение 2,88*x* при использовании 6 вычислительных узлов. Подставляя эти значения в формулу (3.1), получаем долю последовательных операций $f \approx 0{,}04$, то есть всего 4% от общего времени выполнения приходится на нераспараллеливаемые участки алгоритма, что свидетельствует о высоком качестве реализации параллельной части. Аналогично были рассчитаны эти показатели и для других тестируемых размерностей (таблица 3.3).

Таблица 3.3 – Оценка доли последовательного кода (закон Амдала)

Размер	**Узлов (*p*)**	**Ускорение (*S*)**	**Доля последовательного кода (*f*)**	**Предел ускорения (1/*f*)**
500×500	6	4,00*x*	2,5%	40*x*
1000×1000	6	2,88*x*	4,0%	25*x*
1500×1500	6	2,36*x*	5,5%	18*x*
2000×2000	6	1,61*x*	11,0%	9*x*

Для матрицы размером 1000×1000 по закону Густафсона-Барсиса при фиксированном времени выполнения 0,5 секунды было установлено, что на 1 узле решается матрица 700×700, а на 6 узлах – 1000×1000. Подставляя эти значения в формулу (3.2), получаем масштабированное ускорение ≈2,92, что означает увеличение объёма вычислений почти в 3 раза при фиксированном времени. Аналогично были рассчитаны эти показатели и для других тестируемых размерностей (таблица 3.4).

Таблица 3.4 – Масштабирование размера задачи

Количество узлов (*p*)	**Размер матрицы**	**Рост линейного размера**	**Рост объёма вычислений**
1	700×700	1,00*x*	1,00*x*
2	800×800	1,14*x*	1,49*x*
4	900×900	1,29*x*	2,13*x*
6	1000×1000	1,43*x*	2,92*x*
8	1000×1000	1,43*x*	2,92*x*

Увеличение количества узлов с 1 до 6 позволяет за одно и то же время решать матрицу, линейный размер которой вырос в 1,43 раза, а объём вычислений – почти в 3 раза. Дальнейшее увеличение узлов до 8 не даёт прироста, так как накладные расходы на коммуникации начинают превышать выигрыш от распараллеливания. Наилучший результат демонстрирует конфигурация из 6 узлов, обеспечивающая максимальное масштабирование задачи при фиксированном времени выполнения.

3.4.3 Верификация разработанной распределённой системы необходима для подтверждения корректности численного решения систем линейных алгебраических уравнений методом Гаусса. Для обеспечения независимой проверки в качестве эталонного средства была выбрана вычислительная среда *Octave*. Данная система предоставляет алгоритмы линейной алгебры, включая оператор левого деления \, который реализует *LU*-разложение с частичным выбором главного элемента – метод, численно эквивалентный классическому методу Гаусса.

В качестве тестовых данных были выбраны три системы линейных алгебраических уравнений размерностями 100×100, 500×500 и 1000×1000. Для каждой размерности матрица и вектор правой части в виде текстовых файлов загружались как в разработанную распределённую систему, так и в среду *Octave*. В *Octave* решение выполнялось с помощью оператора $x = A \setminus b$.

Алгоритм работы таков:

– загрузка в *Octave* исходных данных;

– решение каждой из систем;

– загрузка решений распределенной системы;

– расчет метрик для оценки качества решения распределенной системы через *Octave* (норма невязки для решения, полученного разработанной системой, относительная погрешность, максимальная абсолютная погрешность).

На рисунках Г.1 Г.2, Г.3, которые находятся в приложении Г, приведены расчеты и сравнения результатов решения СЛАУ размерностями 100×100, 500×500 и 1000×1000 через *Octave* и через разработанную распределённую систему.

Рисунок Г.1 показывает сравнение решений через *Octave* (сверху) и распределенную систему (снизу). Как видно, решения систем в обоих случаях совпадают.

На рисунке Г.2 представлен результат вычисления абсолютной и относительной погрешностей через *Octave* (слева) и распределенной системы (справа). Абсолютная погрешность показывает самое большое расхождение между решением, полученным системой, и эталоном по любой неизвестной. Относительная погрешность показывает, насколько сильно полученное решение отличается от эталона в процентах, независимо от того, большие числа или маленькие. С ростом размерности погрешность немного увеличивается, что объясняется большим количеством выполненных операций и, как следствие, большим накоплением ошибок округления. Однако даже для самой большой матрицы 1000×1000 относительная погрешность не превышает пяти тысячных процента. Как видно, значения находятся также на одном уровне, т.е. даже самое большое расхождение по любой из неизвестных не превышает пяти миллионных долей единицы.

На рисунке Г.3 представлено сравнение коэффициентов невязок для решений через *Octave* (слева) и распределенной системы (справа). Невязка показывает, насколько сильно отличается левая часть уравнения от правой после подстановки найденного решения. Как видно, значения невязки разработанной системы и *Octave* находятся на одном уровне. Это означает, что решения, полученные разработанной системой, полностью корректны.

В таблице 3.5 приведены рассчитанные метрики для решений через *Octave* и разработанной распределенной системы.

Таблица 3.5 – Сравнение верификационных метрик

Размер матрицы	**Невязка (система)**	**Невязка (*Octave*)**	**Абсолютная погрешность**	**Относительная погрешность**
100x100	6.29×10^{-14}	5.21×10^{-14}	4.82×10^{-7}	4.47×10^{-6}
500x500	2.68×10^{-13}	1.88×10^{-13}	4.98×10^{-7}	2.35×10^{-5}
1000x1000	5.98×10^{-13}	3.88×10^{-13}	4.99×10^{-7}	4.88×10^{-5}

При сравнении решений, полученных разработанной распределенной системой с эталонными значениями, полученными при помощи системы *Octave,* выявили, что значения векторов решений *x* совпадают, значения невязок также совпадают и находятся в пределах допустимой нормы. Вычисленные погрешности показывают незначительные расхождения результатов решений. Следовательно, разработанная распределенная система корректно выполняет расчеты.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы была спроектирована, разработана и протестирована распределённая система для решения систем линейных алгебраических уравнений (СЛАУ) методом Гаусса.

Актуальность работы обусловлена необходимостью многократного и быстрого решения больших систем высокой размерности, возникающих в задачах гидродинамики, механики деформируемых тел, оптимизации, машинного обучения и других областях, где последовательное выполнение классических алгоритмов на одном вычислительном узле становится невозможным из-за ограничений по памяти и времени счёта.

При реализации были использованы технологии и инструменты, выбранные в соответствии с начальными требованиями к проекту. В качестве языка программирования был выбран *C#* на платформе *.NET*, обеспечивающий высокую производительность, автоматическое управление памятью и встроенные средства для сетевого взаимодействия. Для реализации распределённых вычислений применена архитектура «*Master-Worker*», в которой координатор управляет вычислительным процессом и распределяет строки матрицы между рабочими узлами по круговому принципу.

Вычислительное ядро системы реализовано с использованием векторных инструкций. Для работы с матрицами большой размерности, превышающими объём оперативной памяти, задействован механизм отображения файлов в память. Сетевое взаимодействие между координатором и воркерами построено на протоколе *TCP/IP*. Пользовательский интерфейс реализован в виде *WPF*-приложения, предоставляющего средства для загрузки матриц и векторов из файлов, запуска расчётов с различным количеством узлов и визуализации результатов в виде диаграмм.

Созданное приложение позволяет решать СЛАУ как в последовательном, так и в распределённом режимах. Пользователь может выбрать количество рабочих узлов, загрузить пакет тестовых систем, просмотреть подробный лог выполнения и проанализировать ускорение от распараллеливания. Система автоматически рассчитывает норму невязки и верифицирует полученное решение. Проведённое тестирование подтвердило корректность работы всех модулей и алгоритмов: норма невязки не превышает машинного допуска, а распределённая версия демонстрирует устойчивое ускорение по сравнению с последовательной.

Достоинствами разработанной системы являются простота развёртывания, возможность масштабирования путём добавления новых узлов, эффективное использование аппаратных ресурсов за счёт векторных инструкций и работа с матрицами, размер которых превышает доступный объём оперативной памяти.

Практическая значимость работы заключается в возможности применения системы для решения прикладных задач вычислительной линейной алгебры в научных и инженерных расчётах. Созданная распределённая система полностью удовлетворяет поставленным требованиям и выполняет все необходимые задачи. Полученное приложение готово к эксплуатации.

Список использованных источников

1. Math.NET Numerics Documentation. – Электрон. данные. – Режим доступа: https://numerics.mathdotnet.com/. – Дата доступа: 07.03.2026.

2. Golub G.H., Van Loan C.F. Matrix Computations. – 4th ed. – Baltimore: Johns Hopkins University Press, 2013. – 784 p.

3. Деммель Дж. Вычислительная линейная алгебра: теория и приложения / пер. с англ. – М.: Мир, 2001. – 432 с.

4. Blackford L.S., Choi J., Cleary A., D'Azevedo E., Demmel J., Dhillon I., Dongarra J.J., Hammarling S., Henry G., Petitet A., Stanley K., Walker D., Whaley R.C. ScaLAPACK Users' Guide. – Philadelphia: SIAM, 1997. – 407 p.

5. Воеводин В.В., Воеводин Вл.В. Параллельные вычисления. – СПб.: БХВ-Петербург, 2002. – 608 с.

6. Таненбаум Э., ван Стеен М. Распределённые системы: принципы и парадигмы / пер. с англ. – СПб.: Питер, 2003. – 880 с.

7. Microsoft Learn: C++ Documentation. – Электрон. данные. – Режим доступа: https://learn.microsoft.com/ru-ru/cpp/. – Дата доступа: 08.03.2026.

8. Официальная документация Python. – Электрон. данные. – Режим доступа: https://docs.python.org/3/. – Дата доступа: 08.03.2026.

9. Oracle Java Documentation. – Электрон. данные. – Режим доступа: https://docs.oracle.com/en/java/. – Дата доступа: 08.03.2026.

10. Microsoft Learn: C# Documentation. – Электрон. данные. – Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/csharp/. – Дата доступа: 08.03.2026.

ПРИЛОЖЕНИЕ А
(обязательное)

Исходный код приложения

MainWindow.xaml.cs

```
using System;
using System.IO;
using System.Linq;
using System.Threading.Tasks;
using System.Windows;
using Microsoft.Win32;
using Gaussian.Shared;
using Gaussian.Coordinator;
using System.Text;
using System.Globalization;
using System.Collections.Generic;
using System.Text.RegularExpressions;
using System.Diagnostics;
using System.Windows.Controls;

namespace Gaussian.Client
{
   public partial class MainWindow : Window
   {
      private readonly SolutionCoordinator _coordinator = new();
      private readonly DataGenerator _generator = new();
      private List<SolveTask> _batchQueue = new List<SolveTask>();
      private List<SolveTask> _testMatrices = new List<SolveTask>();
      private string _singleMatrixPath = "";
      private string _singleVectorPath = "";
      private const string TargetDir = @"D:\RIS_COURSE_PROJECT\Matrices";
      private List<SessionStats> _currentBatchSessions = new List<SessionStats>();
      private List<WorkerBenchmark> _workerBenchmarks = new List<WorkerBenchmark>();

      public MainWindow()
      {
         InitializeComponent();
         StatusCores.Text = $"CPU: {Environment.ProcessorCount} ядер";
         if (!Directory.Exists(TargetDir)) Directory.CreateDirectory(TargetDir);
         Loaded += MainWindow_Loaded;
         BtnSolve.IsEnabled = false;
      }

      private void MainWindow_Loaded(object sender, RoutedEventArgs e)
      {
         ChartCanvas.SizeChanged += (s, ev) => ChartRenderer.DrawComparisonChart(ChartCan-
vas, _currentBatchSessions);
         NodeLoadCanvas.SizeChanged += (s, ev) => ChartRenderer.DrawNodeLoad-
Chart(NodeLoadCanvas, _currentBatchSessions);
```

```
        WorkerChartCanvas.SizeChanged += (s, ev) => ChartRenderer.DrawWorkerChart(Worker-
ChartCanvas, _workerBenchmarks);
    }

    private bool IsMatrixFile(string filePath)
    {
        try
        {
            var lines = File.ReadLines(filePath).Take(5).ToList();
            if (lines.Count == 0) return false;
            int colCount = lines[0].Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEn-
tries).Length;
            int rowCount = 0;
            foreach (var line in lines)
            {
                var parts = line.Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEntries);
                if (parts.Length != colCount) return false;
                rowCount++;
            }
            return rowCount >= 2 && colCount >= 2;
        }
        catch { return false; }
    }

    private bool IsVectorFile(string filePath)
    {
        try
        {
            var lines = File.ReadLines(filePath).Take(10).ToList();
            if (lines.Count == 0) return false;
            foreach (var line in lines)
            {
                var parts = line.Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEntries);
                if (parts.Length != 1) return false;
                if (!double.TryParse(parts[0], NumberStyles.Float, CultureInfo.InvariantCulture, out
_)) return false;
            }
            return true;
        }
        catch { return false; }
    }

    private int GetMatrixSizeFromFile(string filePath)
    {
        try
        {
            var firstLine = File.ReadLines(filePath).FirstOrDefault();
            if (firstLine == null) return 0;
            return firstLine.Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEntries).Length;
        }
        catch { return 0; }
    }
```

```
        private void UpdateLoadedMatricesList()
        {
            Dispatcher.InvokeAsync(() =>
            {
                TxtLoadedMatrices.Clear();
                TxtLoadedMatrices.AppendText($"Загружено матриц: {_testMatrices.Count}\n");
                TxtLoadedMatrices.AppendText("━━━
```

```
            var newTask = new SolveTask
            {
                MatrixPath = mPath,
                VectorPath = vPath,
                Size = currentSize,
                Name = $"matrix_{currentSize}"
            };
            _batchQueue.Add(newTask);
            _testMatrices.Add(newTask);
            TxtLog.AppendText($"готово\n");
        }

        TxtLog.AppendText($"--- Всего в очереди: {_batchQueue.Count} ---\n");
        TxtLog.ScrollToEnd();
        UpdateLoadedMatricesList();
        BtnSolve.IsEnabled = _testMatrices.Count > 0;
    }

    private void SelectMatrix_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog { Filter = "txt|*.txt", InitialDirectory = TargetDir };
        if (dialog.ShowDialog() == true)
        {
            string selectedPath = dialog.FileName;
            if (!IsMatrixFile(selectedPath))
            {
                MessageBox.Show("Ошибка: Выбранный файл не является матрицей.", "Ошибка",
MessageBoxButton.OK, MessageBoxImage.Error);
                return;
            }
            _singleMatrixPath = selectedPath;
            TxtLog.AppendText($"\n[ЗАГРУЗКА] Матрица: {Path.GetFileName(_singleMatrix-
Path)}\n");

            if (!string.IsNullOrEmpty(_singleVectorPath))
            {
                int matrixSize = GetMatrixSizeFromFile(_singleMatrixPath);
                int vectorSize = File.ReadLines(_singleVectorPath).Count();
                if (matrixSize != vectorSize)
                {
                    MessageBox.Show($"Ошибка: Размер матрицы ({matrixSize}) не совпадает с
размером вектора ({vectorSize})", "Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
                    _singleMatrixPath = "";
                    _singleVectorPath = "";
                    return;
                }
                _testMatrices.Clear();
                _testMatrices.Add(new SolveTask { MatrixPath = _singleMatrixPath, VectorPath =
_singleVectorPath, Size = matrixSize, Name = Path.GetFileNameWithoutExtension(_singleMatrix-
Path) });
                UpdateLoadedMatricesList();
                BtnSolve.IsEnabled = true;
```

```
            }
        }
    }

    private void SelectVector_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog { Filter = "txt|*.txt", InitialDirectory = TargetDir };
        if (dialog.ShowDialog() == true)
        {
            string selectedPath = dialog.FileName;
            if (!IsVectorFile(selectedPath))
            {
                MessageBox.Show("Ошибка: Выбранный файл не является вектором.", "Ошибка",
MessageBoxButton.OK, MessageBoxImage.Error);
                return;
            }
            _singleVectorPath = selectedPath;
            TxtLog.AppendText($"\n[ЗАГРУЗКА] Вектор: {Path.GetFileName(_singleVector-
Path)}\n");

            if (!string.IsNullOrEmpty(_singleMatrixPath))
            {
                int matrixSize = GetMatrixSizeFromFile(_singleMatrixPath);
                int vectorSize = File.ReadLines(_singleVectorPath).Count();
                if (matrixSize != vectorSize)
                {
                    MessageBox.Show($"Ошибка: Размер матрицы ({matrixSize}) не совпадает с
размером вектора ({vectorSize})", "Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
                    _singleMatrixPath = "";
                    _singleVectorPath = "";
                    return;
                }
                _testMatrices.Clear();
                _testMatrices.Add(new SolveTask { MatrixPath = _singleMatrixPath, VectorPath =
_singleVectorPath, Size = matrixSize, Name = Path.GetFileNameWithoutExtension(_singleMatrix-
Path) });
                UpdateLoadedMatricesList();
                BtnSolve.IsEnabled = true;
            }
        }
    }

    private void SelectFolder_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFolderDialog();
        if (dialog.ShowDialog() == true)
        {
            var files = Directory.GetFiles(dialog.FolderName, "*.txt");
            var matrices = files.Where(f => Path.GetFileName(f).StartsWith("matrix_")).ToList();

            foreach (var m in matrices)
            {
```

```
                var match = Regex.Match(Path.GetFileName(m), @"matrix_(\d+)");
                if (match.Success)
                {
                    string s = match.Groups[1].Value;
                    var v = files.FirstOrDefault(f => Path.GetFileName(f).Contains($"freeterms_{s}"));
                    if (v != null)
                    {
                        var newTask = new SolveTask { MatrixPath = m, VectorPath = v, Size =
int.Parse(s), Name = Path.GetFileNameWithoutExtension(m) };
                        _batchQueue.Add(newTask);
                        _testMatrices.Add(newTask);
                        TxtLog.AppendText($"\nДобавлена матрица {newTask.Size}x{new-
Task.Size}");
                    }
                }
            }
            TxtLog.AppendText($"\n[ПАПКА] Добавлено задач: {_batchQueue.Count}\n");
            TxtLog.ScrollToEnd();
            UpdateLoadedMatricesList();
            BtnSolve.IsEnabled = _testMatrices.Count > 0;
        }
    }

    private async void BtnSolve_Click(object sender, RoutedEventArgs e)
    {
        List<SolveTask> tasksToRun = _testMatrices.Count > 0 ? new List<SolveTask>(_test-
Matrices) : new List<SolveTask>(_batchQueue);
        if (tasksToRun.Count == 0) return;

        TxtLog.Clear();
        TxtLog.Ap-
pendText("═══════════════════════════════════════════════════════════════════════
═══════════════════════════\n");
        TxtLog.AppendText("                    ЗАПУСК РАСЧЕТА\n");
        TxtLog.Ap-
pendText("═══════════════════════════════════════════════════════════════════════
═══════════════════════════\n\n");

        int workers = int.Parse(((ComboBoxItem)ComboWorkers.SelectedItem).Con-
tent.ToString());
        BtnSolve.IsEnabled = false;
        ProgressIndicator.Visibility = Visibility.Visible;
        _currentBatchSessions.Clear();

        var completedResults = new List<Tuple<SolveTask, SolveResponse>>();

        for (int idx = 0; idx < tasksToRun.Count; idx++)
        {
            var task = tasksToRun[idx];
            TxtLog.AppendText($"\n>>> ОБРАБОТКА: {task.Name}
({task.Size}x{task.Size})...\n");
```

```
            var res = await Task.Run(() => _coordinator.HandleRequest(new SolveRequest { Matrix-
Path = task.MatrixPath, FreeTermsPath = task.VectorPath }, workers));

            completedResults.Add(Tuple.Create(task, res));

            double nodeLoadPercent = (res.ParallelTimeMs / Math.Max(1, res.SequentialTimeMs)) *
100;
            if (nodeLoadPercent > 100) nodeLoadPercent = 100;

            // Если параллельное время больше последовательного, ускорение считаем как 1.0
            double acceleration = res.Acceleration;
            if (res.ParallelTimeMs > res.SequentialTimeMs)
            {
                acceleration = 1.0;
                nodeLoadPercent = 100;
            }

            var session = new SessionStats
            {
                TaskSize = task.Size,
                SequentialTimeMs = res.SequentialTimeMs,
                ParallelTimeMs = res.ParallelTimeMs,
                Acceleration = acceleration,
                Efficiency = (acceleration / workers) * 100,
                Timestamp = DateTime.Now,
                ServiceTimeMs = res.ParallelTimeMs,
                NodeLoadPercent = nodeLoadPercent,
                WorkersCount = workers,
                Residual = res.Residual
            };
            _currentBatchSessions.Add(session);

            TxtLog.AppendText($"\n[РЕЗУЛЬТАТЫ]");
            TxtLog.AppendText($"\n    Невязка: {res.Residual:E3}");
            TxtLog.AppendText($"\n    Время последовательно: {res.SequentialTimeMs / 1000:F3}
c");
            TxtLog.AppendText($"\n    Время параллельно: {res.ParallelTimeMs / 1000:F3} c");
            TxtLog.AppendText($"\n    Ускорение: {acceleration:F2}x");
            TxtLog.AppendText($"\n    Эффективность: {(acceleration / workers) * 100:F1}%");
            TxtLog.AppendText("\n------------------------------------------------------------\n");
            TxtLog.ScrollToEnd();

            await Dispatcher.InvokeAsync(() => UpdateAnalyticsTab());
        }

        TxtLog.Ap-
pendText("\n═══════════════════════════════════════════════════════════════════════════
═══════════════════════════\n");
        TxtLog.AppendText("                    РАСЧЕТ ЗАВЕРШЕН\n");
        TxtLog.Ap-
pendText("═══════════════════════════════════════════════════════════════════════════
═══════════════════════════\n");
```

```
            await Dispatcher.InvokeAsync(() =>
            {
                foreach (var item in completedResults)
                {
                    var task = item.Item1;
                    var res = item.Item2;

                    if (string.IsNullOrEmpty(res.Log) || !res.Log.Contains("ERROR"))
                    {
                        var result = MessageBox.Show(
                            $"Сохранить результаты решения для матрицы {task.Name} в файл?",
                            "Сохранение результатов",
                            MessageBoxButton.YesNo,
                            MessageBoxImage.Question);

                        if (result == MessageBoxResult.Yes)
                        {
                            SaveSolutionToFile(res, task.Name);
                        }
                    }
                }
            });

            BtnSolve.IsEnabled = true;
            ProgressIndicator.Visibility = Visibility.Collapsed;
        }

        private void SaveSolutionToFile(SolveResponse response, string matrixName)
        {
            var saveFileDialog = new SaveFileDialog
            {
                Title = $"Сохранить решение для {matrixName}",
                Filter = "Текстовые файлы (*.txt)|*.txt|Все файлы (*.*)|*.*",
                DefaultExt = "txt",
                FileName = $"solution_{matrixName}_{DateTime.Now:yyyyMMdd_HHmmss}.txt"
            };

            if (saveFileDialog.ShowDialog() == true)
            {
                try
                {
                    using (var writer = new StreamWriter(saveFileDialog.FileName, false, Encod-
ing.UTF8))
                    {
                        writer.WriteLine($"# Решение для матрицы: {matrixName}");
                        writer.WriteLine($"# Дата: {DateTime.Now:yyyy-MM-dd HH:mm:ss}");
                        writer.WriteLine($"# Размер: {response.Size}x{response.Size}");
                        writer.WriteLine($"# Время последовательно: {response.SequentialTimeMs:F2}
мс");
                        writer.WriteLine($"# Время параллельно: {response.ParallelTimeMs:F2} мс");
                        writer.WriteLine($"# Ускорение: {response.Acceleration:F2}x");
```

```
                writer.WriteLine($"# Эффективность: {response.Efficiency:F1}%");
                writer.WriteLine($"# Невязка: {response.Residual:E3}");
                writer.WriteLine("# ------------------------------------------------");

                if (response.Solution != null)
                {
                    for (int i = 0; i < response.Solution.Length; i++)
                    {
                        writer.WriteLine(response.Solution[i].ToString("F12", CultureInfo.Invariant-
Culture));
                    }
                }
            }

            TxtLog.AppendText($"\n[СОХРАНЕНИЕ] Результаты для {matrixName}
сохранены в файл: {Path.GetFileName(saveFileDialog.FileName)}\n");
            TxtLog.ScrollToEnd();
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка при сохранении файла: {ex.Message}", "Ошибка",
MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
}

private async void BtnRunWorkerTest_Click(object sender, RoutedEventArgs e)
{
    List<SolveTask> tasksToTest = _testMatrices.Count > 0 ? new List<SolveTask>(_test-
Matrices) : new List<SolveTask>(_batchQueue);
    if (tasksToTest.Count == 0)
    {
        TxtLog.AppendText("\n[ОШИБКА] Нет загруженных матриц для тестирования\n");
        return;
    }

    TxtLog.Clear();
    TxtLog.Ap-
pendText("═══════════════════════════════════════════════════════════════════
═══════════════════════════\n");
    TxtLog.AppendText("                    ТЕСТ ПРОИЗВОДИТЕЛЬНОСТИ\n");
    TxtLog.Ap-
pendText("═══════════════════════════════════════════════════════════════════
═══════════════════════════\n\n");

    BtnRunWorkerTest.IsEnabled = false;
    ProgressIndicator.Visibility = Visibility.Visible;
    _workerBenchmarks.Clear();

    int[] workerCounts = new int[] { 1, 2, 4, 6, 8 };

    foreach (var task in tasksToTest)
```

```
        {
            TxtLog.AppendText($"\n[ТЕСТ] Матрица: {task.Name} ({task.Size}x{task.Size})\n");

            foreach (int workers in workerCounts)
            {
                TxtLog.AppendText($"  Тест с {workers} узлами... ");
                var res = await Task.Run(() => _coordinator.HandleRequest(new SolveRequest { Ma-
trixPath = task.MatrixPath, FreeTermsPath = task.VectorPath }, workers));

                _workerBenchmarks.Add(new WorkerBenchmark
                {
                    WorkerCount = workers,
                    TimeMs = res.ParallelTimeMs,
                    MatrixSize = task.Size,
                    MatrixName = task.Name
                });

                TxtLog.AppendText($"готово ({res.ParallelTimeMs / 1000:F2} с)\n");
                await Dispatcher.InvokeAsync(() => ChartRenderer.DrawWorkerChart(Worker-
ChartCanvas, _workerBenchmarks));
                await Task.Delay(50);
            }
        }

        TxtLog.AppendText($"\n[ТЕСТ ЗАВЕРШЕН]\n");
        BtnRunWorkerTest.IsEnabled = true;
        ProgressIndicator.Visibility = Visibility.Collapsed;
    }

    private void UpdateAnalyticsTab()
    {
        if (_currentBatchSessions.Count == 0) return;

        double avgSequentialTime = _currentBatchSessions.Average(s => s.SequentialTimeMs);
        double avgParallelTime = _currentBatchSessions.Average(s => s.ParallelTimeMs);
        double avgAcceleration = _currentBatchSessions.Average(s => s.Acceleration);
        double avgNodeLoad = _currentBatchSessions.Average(s => s.NodeLoadPercent);

        double avgServiceTime = _currentBatchSessions.Average(s => s.ServiceTimeMs);

        double totalTime = _currentBatchSessions.Sum(s => s.ServiceTimeMs) / 1000.0;
        double arrivalRate = _currentBatchSessions.Count / Math.Max(0.001, totalTime);
        double serviceRate = 1000.0 / Math.Max(0.001, avgServiceTime);

        Dispatcher.InvokeAsync(() =>
        {
            TxtAvgSeqTime.Text = $"{avgSequentialTime / 1000:F2} с";
            TxtAvgParTime.Text = $"{avgParallelTime / 1000:F2} с";
            TxtAvgAccel.Text = $"{avgAcceleration:F2}x";
            TxtAvgNodeLoad.Text = $"{avgNodeLoad:F1}%";
            NodeLoadProgress.Value = avgNodeLoad;
```

```
            TxtAvgServiceTime.Text = $"{avgServiceTime:F0} мс";
            TxtArrivalRate.Text = $"{arrivalRate:F3}";
            TxtServiceRate.Text = $"{serviceRate:F3}";
        });

        ChartRenderer.DrawComparisonChart(ChartCanvas, _currentBatchSessions);
        ChartRenderer.DrawNodeLoadChart(NodeLoadCanvas, _currentBatchSessions);
    }

    private void SliderDensity_ValueChanged(object sender, RoutedPropertyChangedEven-
tArgs<double> e)
    {
        int roundedValue = (int)Math.Round(e.NewValue);
        LblDensityValue.Text = $"{roundedValue}%";
        SliderDensity.Value = roundedValue;
    }

    private void Window_Closed(object sender, EventArgs e)
    {
        foreach (var p in Process.GetProcessesByName("Gaussian.Worker"))
            try { p.Kill(); } catch { }
        _coordinator.Shutdown();
    }
  }
}
```

ChartRenderer.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Shapes;

using Gaussian.Shared;

namespace Gaussian.Client
{
  public static class ChartRenderer
  {
    private static Color[] _matrixColors = new Color[]
    {
        Color.FromRgb(67, 97, 238),
        Color.FromRgb(16, 185, 129),
        Color.FromRgb(245, 158, 11),
        Color.FromRgb(239, 68, 68),
        Color.FromRgb(139, 92, 246),
        Color.FromRgb(236, 72, 153),
        Color.FromRgb(14, 165, 233),
        Color.FromRgb(168, 85, 247),
        Color.FromRgb(234, 179, 8),
```

```
            Color.FromRgb(34, 197, 94)
        };

        public static void DrawComparisonChart(Canvas canvas, List<SessionStats> sessions)
        {
            if (sessions == null || sessions.Count == 0 || canvas.ActualWidth <= 0 || canvas.ActualHeight
<= 0) return;

            canvas.Children.Clear();

            double maxTime = Math.Max(sessions.Max(s => s.SequentialTimeMs), sessions.Max(s =>
s.ParallelTimeMs));
            if (maxTime <= 0) maxTime = 1;

            double chartWidth = canvas.ActualWidth - 120;
            double chartHeight = canvas.ActualHeight - 100;
            if (chartWidth <= 0 || chartHeight <= 0) return;

            DrawGridLines(canvas, chartWidth, chartHeight, maxTime);

            double barWidth = Math.Min(30, chartWidth / (sessions.Count * 3));
            double startX = 70;
            double bottomY = canvas.ActualHeight - 70;

            for (int i = 0; i < sessions.Count; i++)
            {
                Color color = _matrixColors[i % _matrixColors.Length];
                double x = startX + i * (barWidth * 2.5);
                double seqHeight = (sessions[i].SequentialTimeMs / maxTime) * chartHeight;
                double parHeight = (sessions[i].ParallelTimeMs / maxTime) * chartHeight;

                Color seqColor = Color.FromRgb((byte)(color.R * 0.6), (byte)(color.G * 0.6),
(byte)(color.B * 0.6));

                if (seqHeight > 0)
                {
                    Rectangle seqBar = new Rectangle { Width = barWidth, Height = seqHeight, Fill =
new SolidColorBrush(seqColor) };
                    Canvas.SetTop(seqBar, bottomY - seqHeight);
                    Canvas.SetLeft(seqBar, x);
                    canvas.Children.Add(seqBar);
                }

                if (parHeight > 0)
                {
                    Rectangle parBar = new Rectangle { Width = barWidth, Height = parHeight, Fill =
new SolidColorBrush(color) };
                    Canvas.SetTop(parBar, bottomY - parHeight);
                    Canvas.SetLeft(parBar, x + barWidth + 5);
                    canvas.Children.Add(parBar);
                }
```

```
            TextBlock label = new TextBlock
            {
                Text = sessions[i].TaskSize.ToString(),
                FontSize = 9,
                FontWeight = System.Windows.FontWeights.Bold,
                Foreground = new SolidColorBrush(Color.FromRgb(226, 232, 240))
            };
            Canvas.SetTop(label, bottomY + 5);
            Canvas.SetLeft(label, x + barWidth / 2);
            canvas.Children.Add(label);
        }

        AddLegend(canvas, sessions, barWidth, startX, bottomY);
    }

    public static void DrawNodeLoadChart(Canvas canvas, List<SessionStats> sessions)
    {
        if (sessions == null || sessions.Count == 0 || canvas.ActualWidth <= 0 || canvas.ActualHeight
<= 0) return;

        canvas.Children.Clear();

        double chartWidth = canvas.ActualWidth - 100;
        double chartHeight = canvas.ActualHeight - 70;
        if (chartWidth <= 0 || chartHeight <= 0) return;

        for (int i = 0; i <= 100; i += 20)
        {
            double y = canvas.ActualHeight - 50 - (i / 100.0) * chartHeight;
            Line gridLine = new Line
            {
                X1 = 60,
                Y1 = y,
                X2 = canvas.ActualWidth - 20,
                Y2 = y,
                Stroke = new SolidColorBrush(Color.FromArgb(40, 148, 163, 184)),
                StrokeThickness = 0.5
            };
            canvas.Children.Add(gridLine);

            TextBlock gridLabel = new TextBlock
            {
                Text = $"{i}%",
                FontSize = 8,
                Foreground = new SolidColorBrush(Color.FromRgb(100, 116, 139))
            };
            Canvas.SetLeft(gridLabel, 10);
            Canvas.SetTop(gridLabel, y - 8);
            canvas.Children.Add(gridLabel);
        }

        double barWidth = Math.Min(40, chartWidth / (sessions.Count * 1.5));
```

```
double startX = 70;
double bottomY = canvas.ActualHeight - 50;

for (int i = 0; i < sessions.Count; i++)
{
    Color color = _matrixColors[i % _matrixColors.Length];
    double x = startX + i * (barWidth + 15);
    double loadPercent = sessions[i].NodeLoadPercent;
    double barHeight = (loadPercent / 100.0) * chartHeight;

    if (barHeight > 0)
    {
        Rectangle loadBar = new Rectangle
        {
            Width = barWidth,
            Height = barHeight,
            Fill = new SolidColorBrush(color)
        };
        Canvas.SetTop(loadBar, bottomY - barHeight);
        Canvas.SetLeft(loadBar, x);
        canvas.Children.Add(loadBar);

        TextBlock percentLabel = new TextBlock
        {
            Text = $"{loadPercent:F0}%",
            FontSize = 9,
            FontWeight = System.Windows.FontWeights.Bold,
            Foreground = new SolidColorBrush(color)
        };
        Canvas.SetTop(percentLabel, bottomY - barHeight - 18);
        Canvas.SetLeft(percentLabel, x + barWidth / 4);
        canvas.Children.Add(percentLabel);
    }

    TextBlock label = new TextBlock
    {
        Text = sessions[i].TaskSize.ToString(),
        FontSize = 9,
        FontWeight = System.Windows.FontWeights.Bold,
        Foreground = new SolidColorBrush(Color.FromRgb(226, 232, 240))
    };
    Canvas.SetTop(label, bottomY + 5);
    Canvas.SetLeft(label, x + barWidth / 4);
    canvas.Children.Add(label);
}

TextBlock xLabel = new TextBlock
{
    Text = "Размер матрицы",
    FontSize = 10,
    Foreground = new SolidColorBrush(Color.FromRgb(148, 163, 184))
};
```

```
            Canvas.SetLeft(xLabel, canvas.ActualWidth / 2 - 50);
            Canvas.SetBottom(xLabel, 10);
            canvas.Children.Add(xLabel);
        }

        public static void DrawWorkerChart(Canvas canvas, List<WorkerBenchmark> benchmarks)
        {
            if (benchmarks == null || benchmarks.Count == 0 || canvas.ActualWidth <= 0 || canvas.Actu-
alHeight <= 0) return;

            canvas.Children.Clear();

            var groupedByMatrix = benchmarks.GroupBy(b => b.MatrixName).ToList();
            double maxTime = benchmarks.Max(b => b.TimeMs);
            if (maxTime <= 0) maxTime = 1;

            double chartWidth = canvas.ActualWidth - 120;
            double chartHeight = canvas.ActualHeight - 100;
            if (chartWidth <= 0 || chartHeight <= 0) return;

            for (int i = 0; i <= 5; i++)
            {
                double y = canvas.ActualHeight - 70 - (i * chartHeight / 5);
                double val = maxTime * i / 5 / 1000;

                Line gridLine = new Line
                {
                    X1 = 70,
                    Y1 = y,
                    X2 = canvas.ActualWidth - 30,
                    Y2 = y,
                    Stroke = new SolidColorBrush(Color.FromArgb(40, 148, 163, 184)),
                    StrokeThickness = 0.5
                };
                canvas.Children.Add(gridLine);

                TextBlock gridLabel = new TextBlock
                {
                    Text = $"{val:F1}c",
                    FontSize = 8,
                    Foreground = new SolidColorBrush(Color.FromRgb(100, 116, 139))
                };
                Canvas.SetLeft(gridLabel, 10);
                Canvas.SetTop(gridLabel, y - 8);
                canvas.Children.Add(gridLabel);
            }

            for (int mIdx = 0; mIdx < groupedByMatrix.Count; mIdx++)
            {
                var matrix = groupedByMatrix[mIdx];
                Color color = _matrixColors[mIdx % _matrixColors.Length];
                var sortedData = matrix.OrderBy(b => b.WorkerCount).ToList();
```

```
            Polyline line = new Polyline { Stroke = new SolidColorBrush(color), StrokeThickness =
2.5 };

            for (int i = 0; i < sortedData.Count; i++)
            {
                double x = 70 + (i * chartWidth / (sortedData.Count - 1));
                double y = canvas.ActualHeight - 70 - (sortedData[i].TimeMs / maxTime) * chart-
Height;
                line.Points.Add(new Point(x, y));

                Ellipse point = new Ellipse
                {
                    Width = 8,
                    Height = 8,
                    Fill = new SolidColorBrush(color),
                    Stroke = new SolidColorBrush(Colors.White),
                    StrokeThickness = 1.5
                };
                Canvas.SetLeft(point, x - 4);
                Canvas.SetTop(point, y - 4);
                canvas.Children.Add(point);
            }
            canvas.Children.Add(line);
        }

        int[] workerCounts = new int[] { 1, 2, 4, 6, 8 };
        for (int i = 0; i < workerCounts.Length; i++)
        {
            double x = 70 + (i * chartWidth / (workerCounts.Length - 1));
            TextBlock workerLabel = new TextBlock
            {
                Text = workerCounts[i].ToString(),
                FontSize = 9,
                FontWeight = FontWeights.Bold,
                Foreground = new SolidColorBrush(Color.FromRgb(226, 232, 240))
            };
            Canvas.SetTop(workerLabel, canvas.ActualHeight - 45);
            Canvas.SetLeft(workerLabel, x - 8);
            canvas.Children.Add(workerLabel);
        }

        TextBlock xLabel = new TextBlock
        {
            Text = "Количество узлов (воркеров)",
            FontSize = 11,
            FontWeight = FontWeights.Bold,
            Foreground = new SolidColorBrush(Color.FromRgb(148, 163, 184))
        };
        Canvas.SetLeft(xLabel, canvas.ActualWidth / 2 - 80);
        Canvas.SetBottom(xLabel, 10);
        canvas.Children.Add(xLabel);
```

```
            TextBlock yLabel = new TextBlock
            {
                Text = "Время выполнения (секунды)",
                FontSize = 11,
                FontWeight = FontWeights.Bold,
                Foreground = new SolidColorBrush(Color.FromRgb(148, 163, 184))
            };
            double yLabelX = 15;
            double yLabelY = canvas.ActualHeight / 2 - 40;
            Canvas.SetLeft(yLabel, yLabelX);
            Canvas.SetTop(yLabel, yLabelY);
            RotateTransform rotate = new RotateTransform(-90, 0, yLabelY + 10);
            yLabel.RenderTransform = rotate;
            canvas.Children.Add(yLabel);

            TextBlock title = new TextBlock
            {
                FontSize = 12,
                FontWeight = FontWeights.Bold,
                Foreground = new SolidColorBrush(Color.FromRgb(248, 250, 252))
            };
            Canvas.SetLeft(title, canvas.ActualWidth / 2 - 150);
            Canvas.SetBottom(title, canvas.ActualHeight - 10);
            canvas.Children.Add(title);

            double legendY = 15;
            for (int mIdx = 0; mIdx < Math.Min(groupedByMatrix.Count, 6); mIdx++)
            {
                var matrix = groupedByMatrix[mIdx];
                Color color = _matrixColors[mIdx % _matrixColors.Length];

                Rectangle legendRect = new Rectangle { Width = 12, Height = 12, Fill = new SolidColor-
Brush(color) };
                Canvas.SetTop(legendRect, legendY + (mIdx * 20));
                Canvas.SetLeft(legendRect, canvas.ActualWidth - 150);
                canvas.Children.Add(legendRect);

                TextBlock legendText = new TextBlock
                {
                    Text = matrix.Key,
                    FontSize = 8,
                    Foreground = new SolidColorBrush(Color.FromRgb(203, 213, 225))
                };
                Canvas.SetTop(legendText, legendY + (mIdx * 20) - 2);
                Canvas.SetLeft(legendText, canvas.ActualWidth - 133);
                canvas.Children.Add(legendText);
            }
        }

        private static void DrawGridLines(Canvas canvas, double width, double height, double
maxValue)
```

```
        {
            for (int i = 0; i <= 5; i++)
            {
                double y = canvas.ActualHeight - 50 - (i * height / 5);
                double val = maxValue * i / 5 / 1000;

                Line gridLine = new Line
                {
                    X1 = 60,
                    Y1 = y,
                    X2 = canvas.ActualWidth - 20,
                    Y2 = y,
                    Stroke = new SolidColorBrush(Color.FromArgb(40, 148, 163, 184)),
                    StrokeThickness = 0.5
                };
                canvas.Children.Add(gridLine);

                TextBlock gridLabel = new TextBlock
                {
                    Text = $"{val:F1}c",
                    FontSize = 8,
                    Foreground = new SolidColorBrush(Color.FromRgb(100, 116, 139))
                };
                Canvas.SetLeft(gridLabel, 10);
                Canvas.SetTop(gridLabel, y - 8);
                canvas.Children.Add(gridLabel);
            }
        }

        private static void AddLegend(Canvas canvas, List<SessionStats> sessions, double barWidth,
double startX, double bottomY)
        {
            double legendY = canvas.ActualHeight - 20;

            for (int i = 0; i < Math.Min(sessions.Count, 5); i++)
            {
                Color color = _matrixColors[i % _matrixColors.Length];
                double legendX = startX + i * (barWidth * 2.5);

                Rectangle parRect = new Rectangle { Width = 8, Height = 8, Fill = new SolidColor-
Brush(color) };
                Canvas.SetTop(parRect, legendY);
                Canvas.SetLeft(parRect, legendX);
                canvas.Children.Add(parRect);

                Color seqColor = Color.FromRgb((byte)(color.R * 0.6), (byte)(color.G * 0.6),
(byte)(color.B * 0.6));
                Rectangle seqRect = new Rectangle { Width = 8, Height = 8, Fill = new SolidColor-
Brush(seqColor) };
                Canvas.SetTop(seqRect, legendY);
                Canvas.SetLeft(seqRect, legendX + 10);
                canvas.Children.Add(seqRect);
```

```
            TextBlock legendText = new TextBlock
            {
                Text = sessions[i].TaskSize.ToString(),
                FontSize = 8,
                Foreground = new SolidColorBrush(Color.FromRgb(203, 213, 225))
            };
            Canvas.SetTop(legendText, legendY - 2);
            Canvas.SetLeft(legendText, legendX + 22);
            canvas.Children.Add(legendText);
        }
    }
  }
}
```

DataGenerator.cs

```
using System;
using Gaussian.Shared;

namespace Gaussian.Coordinator
{
  public class DataGenerator
  {
    private Random _random = new Random();

    public (SparseMatrix, double[]) GenerateRandomSystem(int size, double density)
    {
        var matrix = new SparseMatrix(size);
        var vector = new double[size];

        for (int i = 0; i < size; i++)
        {
            for (int j = 0; j < size; j++)
            {
                if (_random.NextDouble() < density || i == j)
                {
                    double value = (_random.NextDouble() * 200) - 100;
                    if (Math.Abs(value) > 1e-14)
                    {
                        matrix.Rows[i].Add(new MatrixElement { Column = j, Value = value });
                    }
                }
            }
            vector[i] = (_random.NextDouble() * 200) - 100;
        }

        return (matrix, vector);
    }
  }
}
```

DataManager.cs

```
using System;
using System.IO;
using System.Collections.Generic;
using System.Globalization;
using Gaussian.Shared;

namespace Gaussian.Coordinator
{
    public static class DataManager
    {
        public static (SparseMatrix, double[]) LoadFiles(string matrixPath, string vectorPath)
        {
            var bLines = File.ReadAllLines(vectorPath);
            int n = bLines.Length;
            double[] b = new double[n];

            for (int i = 0; i < n; i++)
                b[i] = double.Parse(bLines[i].Trim(), CultureInfo.InvariantCulture);

            var matrix = new SparseMatrix(n);
            int rowIdx = 0;

            foreach (var line in File.ReadLines(matrixPath))
            {
                if (string.IsNullOrWhiteSpace(line)) continue;
                var parts = line.Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEntries);
                for (int colIdx = 0; colIdx < parts.Length; colIdx++)
                {
                    double val = double.Parse(parts[colIdx], CultureInfo.InvariantCulture);
                    if (Math.Abs(val) > 1e-15)
                        matrix.Rows[rowIdx].Add(new MatrixElement { Column = colIdx, Value = val });
                }
                rowIdx++;
            }

            return (matrix, b);
        }
    }
}
```

SolutionCoordinator.cs

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.IO;
using System.Net.Sockets;
using System.Threading;
using Gaussian.Shared;
using Gaussian.Solver;
```

```
namespace Gaussian.Coordinator
{
    public class SolutionCoordinator
    {
        private List<Process> _workerProcesses = new List<Process>();
        private int _nextPort = 9000;

        public SolveResponse HandleRequest(SolveRequest request, int workersCount)
        {
            var response = new SolveResponse();

            try
            {
                var (matrix, b) = DataManager.LoadFiles(request.MatrixPath, request.FreeTermsPath);
                int n = matrix.Size;
                response.Size = n;

                var engine = new GaussEngine();
                double seqTime;
                response.Solution = engine.SolveSequential(matrix, b, out seqTime);
                response.SequentialTimeMs = seqTime;

                int basePort = _nextPort;
                _nextPort += workersCount + 5;

                for (int i = 0; i < workersCount; i++)
                {
                    var psi = new ProcessStartInfo
                    {
                        FileName = "Gaussian.Worker.exe",
                        Arguments = $"{basePort + i} {n} {i} {workersCount}",
                        UseShellExecute = false,
                        CreateNoWindow = false
                    };
                    _workerProcesses.Add(Process.Start(psi));
                }

                Thread.Sleep(1500);

                var clients = new List<TcpClient>();
                var writers = new List<BinaryWriter>();
                var readers = new List<BinaryReader>();

                for (int i = 0; i < workersCount; i++)
                {
                    var client = new TcpClient();
                    client.Connect("127.0.0.1", basePort + i);
                    clients.Add(client);
                    writers.Add(new BinaryWriter(client.GetStream()));
                    readers.Add(new BinaryReader(client.GetStream()));
                }
```

```
double[,] denseMatrix = new double[n, n];
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        denseMatrix[i, j] = 0;
    }
    if (matrix.Rows[i] != null)
    {
        foreach (var el in matrix.Rows[i])
        {
            denseMatrix[i, el.Column] = el.Value;
        }
    }
}

for (int w = 0; w < workersCount; w++)
{
    for (int row = w; row < n; row += workersCount)
    {
        for (int j = 0; j < n; j++)
        {
            writers[w].Write(denseMatrix[row, j]);
        }
        writers[w].Write(b[row]);
    }
    writers[w].Flush();
}

var parallelSw = Stopwatch.StartNew();

double[] pivotRow = new double[n];
double[] bCopy = (double[])b.Clone();
double[,] aCopy = (double[,])denseMatrix.Clone();

for (int k = 0; k < n; k++)
{
    int pivot = k;
    double maxV = Math.Abs(aCopy[k, k]);
    for (int i = k + 1; i < n; i++)
    {
        double v = Math.Abs(aCopy[i, k]);
        if (v > maxV)
        {
            maxV = v;
            pivot = i;
        }
    }

    if (pivot != k)
    {
        for (int j = k; j < n; j++)
```

```
        {
            double temp = aCopy[k, j];
            aCopy[k, j] = aCopy[pivot, j];
            aCopy[pivot, j] = temp;
        }
        double tb = bCopy[k];
        bCopy[k] = bCopy[pivot];
        bCopy[pivot] = tb;
    }

    for (int j = 0; j < n; j++)
    {
        pivotRow[j] = aCopy[k, j];
    }
    double pivotB = bCopy[k];

    for (int w = 0; w < workersCount; w++)
    {
        for (int j = 0; j < n; j++)
        {
            writers[w].Write(pivotRow[j]);
        }
        writers[w].Write(pivotB);
        writers[w].Write(k);
        writers[w].Flush();
    }

    for (int w = 0; w < workersCount; w++)
    {
        readers[w].ReadBoolean();
    }

    for (int i = k + 1; i < n; i++)
    {
        double factor = aCopy[i, k] / pivotRow[k];
        if (Math.Abs(factor) > 1e-18)
        {
            for (int j = k; j < n; j++)
            {
                aCopy[i, j] -= factor * pivotRow[j];
            }
            bCopy[i] -= factor * pivotB;
        }
    }
}

double[] parallelSolution = new double[n];

for (int i = n - 1; i >= 0; i--)
{
    int owner = i % workersCount;
    int idx = readers[owner].ReadInt32();
```

```
            double val = readers[owner].ReadDouble();
            parallelSolution[idx] = val;

            for (int w = 0; w < workersCount; w++)
            {
                if (w != owner)
                {
                    writers[w].Write(idx);
                    writers[w].Write(val);
                    writers[w].Flush();
                }
            }
        }

        for (int w = 0; w < workersCount; w++)
        {
            readers[w].ReadInt32();
        }

        parallelSw.Stop();
        response.ParallelTimeMs = parallelSw.Elapsed.TotalMilliseconds;
        response.Solution = parallelSolution;
        response.Acceleration = response.SequentialTimeMs / Math.Max(0.001, response.Paral-
lelTimeMs);
        response.Efficiency = (response.Acceleration / workersCount) * 100;
        response.TheoreticalMax = workersCount;
        response.Cores = Environment.ProcessorCount;

        double residual = 0;
        for (int i = 0; i < n; i++)
        {
            double sum = 0;
            for (int j = 0; j < n; j++)
            {
                sum += denseMatrix[i, j] * parallelSolution[j];
            }
            double diff = Math.Abs(sum - b[i]);
            if (diff > residual) residual = diff;
        }
        response.Residual = residual;

        foreach (var w in writers) w.Close();
        foreach (var r in readers) r.Close();
        foreach (var c in clients) c.Close();

        foreach (var p in _workerProcesses)
        {
            try { p.Kill(); } catch { }
        }
        _workerProcesses.Clear();

        response.Log = "OK";
```

```
            }
            catch (Exception ex)
            {
                response.Log = $"ERROR: {ex.Message}";
            }

            return response;
        }

        public void Shutdown()
        {
            foreach (var p in _workerProcesses)
            {
                try { p.Kill(); } catch { }
            }
            _workerProcesses.Clear();
        }
    }
}
```

AnalysisResult.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Gaussian.Shared
{
    public class AnalysisResult
    {
        public int Size { get; set; }
        public double SeqTime { get; set; }
        public double ParTime { get; set; }
        public double Acceleration { get; set; }
        public double Efficiency { get; set; }
        public double TheoreticalMax { get; set; }
    }
}
```

MatrixElement.cs

```
using System;
using System.Collections.Generic;

namespace Gaussian.Shared
{
    public class MatrixElement {
        public int Column { get; set; }
        public double Value { get; set; } }

}
```

SessionStats.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Gaussian.Shared
{
    public class SessionStats
    {
        public int TaskSize { get; set; }
        public double SequentialTimeMs { get; set; }
        public double ParallelTimeMs { get; set; }
        public double Acceleration { get; set; }
        public double Efficiency { get; set; }
        public DateTime Timestamp { get; set; }
        public double ArrivalRate { get; set; }
        public double ServiceRate { get; set; }
        public double ServiceTimeMs { get; set; }
        public double NodeLoadPercent { get; set; }
        public int WorkersCount { get; set; }
        public double Residual { get; set; }
    }
}
```

SolveRequest.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Gaussian.Shared
{
    public class SolveRequest {
        public string MatrixPath { get; set; } = "";
        public string FreeTermsPath { get; set; } = ""; }
}
```

SolveResponse.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Gaussian.Shared
{
    public class SolveResponse
    {
```

```
        public double[]? Solution { get; set; }
        public int Size { get; set; }
        public double ExecutionTimeMs { get; set; }
        public double SequentialTimeMs { get; set; }
        public double ParallelTimeMs { get; set; }
        public double Acceleration { get; set; }
        public double Efficiency { get; set; }
        public double TheoreticalMax { get; set; }
        public int Cores { get; set; }
        public double MemoryUsageMb { get; set; }
        public double Residual { get; set; }
        public string Log { get; set; } = "";
    }
}
```

SolveTask.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Gaussian.Shared
{
    public class SolveTask
    {
        public string MatrixPath { get; set; } = "";
        public string VectorPath { get; set; } = "";
        public int Size { get; set; }
        public string Name { get; set; } = "";
    }
}
```

SparseMatrix.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Gaussian.Shared
{
    public class SparseMatrix
    {
        public int Size { get; set; }
        public List<MatrixElement>[] Rows { get; set; }

        public SparseMatrix(int size)
        {
            Size = size;
            Rows = new List<MatrixElement>[size];
```

```
            for (int i = 0; i < size; i++) Rows[i] = new List<MatrixElement>();
        }

        public SparseMatrix Clone()
        {
            var clone = new SparseMatrix(Size);
            for (int i = 0; i < Size; i++)
            {
                clone.Rows[i] = new List<MatrixElement>(Rows[i].Count);
                foreach (var item in Rows[i]) clone.Rows[i].Add(new MatrixElement { Column =
item.Column, Value = item.Value });
            }
            return clone;
        }
    }
}
```

WorkerBenchmark.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Gaussian.Shared
{
    public class WorkerBenchmark
    {
        public int WorkerCount { get; set; }
        public double TimeMs { get; set; }
        public int MatrixSize { get; set; }
        public string MatrixName { get; set; } = "";
    }
}
```

GaussEngine.cs

```
using System;
using System.IO.MemoryMappedFiles;
using System.Diagnostics;
using Gaussian.Shared;

namespace Gaussian.Solver
{
    public class GaussEngine
    {
        private const double EPS = 1e-18;

        public unsafe double[] SolveSequential(SparseMatrix matrix, double[] b, out double ms)
        {
            int n = matrix.Size;
            double[] a = new double[n * n];
```

```
double[] bCopy = (double[])b.Clone();

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        a[i * n + j] = 0;
    }
    if (matrix.Rows[i] != null)
    {
        foreach (var el in matrix.Rows[i])
        {
            a[i * n + el.Column] = el.Value;
        }
    }
}

var sw = Stopwatch.StartNew();


for (int i = 0; i < n; i++)
{
    int pivot = i;
    double maxV = Math.Abs(a[i * n + i]);
    for (int k = i + 1; k < n; k++)
    {
        double v = Math.Abs(a[k * n + i]);
        if (v > maxV)
        {
            maxV = v;
            pivot = k;
        }
    }

    if (pivot != i)
    {
        for (int j = i; j < n; j++)
        {
            double temp = a[i * n + j];
            a[i * n + j] = a[pivot * n + j];
            a[pivot * n + j] = temp;
        }
        double tb = bCopy[i];
        bCopy[i] = bCopy[pivot];
        bCopy[pivot] = tb;
    }

    double diag = a[i * n + i];
    if (Math.Abs(diag) < EPS) continue;

    for (int j = i + 1; j < n; j++)
    {
```

```
                double factor = a[j * n + i] / diag;
                if (Math.Abs(factor) < EPS) continue;

                for (int k = i; k < n; k++)
                {
                    a[j * n + k] -= factor * a[i * n + k];
                }
                bCopy[j] -= factor * bCopy[i];
            }
        }

        double[] x = new double[n];
        for (int i = n - 1; i >= 0; i--)
        {
            double sum = 0;
            for (int j = i + 1; j < n; j++)
            {
                sum += a[i * n + j] * x[j];
            }
            x[i] = (bCopy[i] - sum) / a[i * n + i];
        }

        sw.Stop();
        ms = sw.Elapsed.TotalMilliseconds;
        return x;
    }

    public double[] SolveParallel(SparseMatrix matrix, double[] b, int threads, out double ms)
    {

        return SolveSequential(matrix, b, out ms);
    }
  }
}
```

Worker.cs

```
using System;
using System.Net.Sockets;
using System.IO;
using System.Threading;

namespace Gaussian.Worker
{
  class Worker
  {
    static int Main(string[] args)
    {
        try
        {
            int port = int.Parse(args[0]);
            int n = int.Parse(args[1]);
            int rank = int.Parse(args[2]);
```

```
            int totalWorkers = int.Parse(args[3]);

            Console.WriteLine($"[{rank}] ЗАПУСК: порт={port}, размер={n}, воркеров={total-
Workers}");
            Console.Out.Flush();

            int rowsPerWorker = (n + totalWorkers - 1) / totalWorkers;
            int[] assignedRows = new int[rowsPerWorker];
            int count = 0;
            for (int i = rank; i < n; i += totalWorkers)
            {
                assignedRows[count++] = i;
            }

            Console.WriteLine($"[{rank}] Назначено строк: {rowsPerWorker}, первые строки:
{string.Join(",", assignedRows.Length > 5 ? assignedRows[..5] : assignedRows)}...");
            Console.Out.Flush();

            var listener = new TcpListener(System.Net.IPAddress.Loopback, port);
            listener.Start();
            Console.WriteLine($"[{rank}] Ожидание подключения на порту {port}...");
            Console.Out.Flush();

            var client = listener.AcceptTcpClient();
            var stream = client.GetStream();
            var reader = new BinaryReader(stream);
            var writer = new BinaryWriter(stream);

            Console.WriteLine($"[{rank}] Подключен к координатору");
            Console.Out.Flush();

            double[][] localRows = new double[rowsPerWorker][];
            double[] localB = new double[rowsPerWorker];

            Console.WriteLine($"[{rank}] Прием локальных данных...");
            Console.Out.Flush();

            for (int idx = 0; idx < rowsPerWorker; idx++)
            {
                localRows[idx] = new double[n];
                for (int j = 0; j < n; j++)
                {
                    localRows[idx][j] = reader.ReadDouble();
                }
                localB[idx] = reader.ReadDouble();

                if (idx % Math.Max(1, rowsPerWorker / 10) == 0 || idx == rowsPerWorker - 1)
                {
                    Console.WriteLine($"[{rank}] Принята строка {idx + 1}/{rowsPerWorker}
(глобальная строка {assignedRows[idx]})");
                    Console.Out.Flush();
                }
```

```
        }

        Console.WriteLine($"[{rank}] Начало прямого хода (исключение Гаусса)...");
        Console.Out.Flush();

        for (int k = 0; k < n; k++)
        {
            double[] pivotRow = new double[n];
            double pivotB = 0;
            int pivotRowIndex = -1;

            for (int j = 0; j < n; j++)
            {
                pivotRow[j] = reader.ReadDouble();
            }
            pivotB = reader.ReadDouble();
            pivotRowIndex = reader.ReadInt32();

            if (k % Math.Max(1, n / 20) == 0 || k == n - 1)
            {
                Console.WriteLine($"[{rank}] Прямой ход: итерация {k + 1}/{n}, pivot
строка={pivotRowIndex}");
                Console.Out.Flush();
            }

            int eliminationCount = 0;
            for (int idx = 0; idx < rowsPerWorker; idx++)
            {
                if (assignedRows[idx] > pivotRowIndex)
                {
                    double factor = localRows[idx][k] / pivotRow[k];
                    if (Math.Abs(factor) > 1e-18)
                    {
                        for (int j = k; j < n; j++)
                        {
                            localRows[idx][j] -= factor * pivotRow[j];
                        }
                        localB[idx] -= factor * pivotB;
                        eliminationCount++;
                    }
                }
            }

            writer.Write(true);
            writer.Flush();

            if (eliminationCount > 0 && k % Math.Max(1, n / 20) == 0)
            {
                Console.WriteLine($"[{rank}]   исключено строк: {eliminationCount}");
                Console.Out.Flush();
            }
        }
```

```
Console.WriteLine($"[{rank}] Прямой ход завершен. Начало обратного хода...");
Console.Out.Flush();

double[] solution = new double[n];

for (int i = n - 1; i >= 0; i--)
{
    bool ownsRow = false;
    int localIdx = -1;
    for (int idx = 0; idx < rowsPerWorker; idx++)
    {
        if (assignedRows[idx] == i)
        {
            ownsRow = true;
            localIdx = idx;
            break;
        }
    }

    if (ownsRow)
    {
        double sum = 0;
        for (int j = i + 1; j < n; j++)
        {
            sum += localRows[localIdx][j] * solution[j];
        }
        solution[i] = (localB[localIdx] - sum) / localRows[localIdx][i];

        writer.Write(i);
        writer.Write(solution[i]);
        writer.Flush();

        if (i % Math.Max(1, n / 20) == 0 || i == 0)
        {
            Console.WriteLine($"[{rank}] Обратный ход: вычислено x[{i}] = {solu-
tion[i]:F6}");
            Console.Out.Flush();
        }
    }
    else
    {
        int idx = reader.ReadInt32();
        double val = reader.ReadDouble();
        solution[idx] = val;
    }
}

writer.Write(-1);
writer.Flush();

Console.WriteLine($"[{rank}] Решение получено. Завершение работы...");
```

```
            Console.Out.Flush();

            reader.Close();
            writer.Close();
            stream.Close();
            client.Close();
            listener.Stop();

            Console.WriteLine($"[{rank}] Воркер успешно завершен");
            Console.Out.Flush();

            return 0;
        }
        catch (Exception ex)
        {
            Console.WriteLine($"[ERROR] {ex.Message}");
            Console.Out.Flush();
            return 1;
        }
    }
  }
}
```

ПРИЛОЖЕНИЕ Б
(обязательное)

Экранные копии графического интерфейса приложения

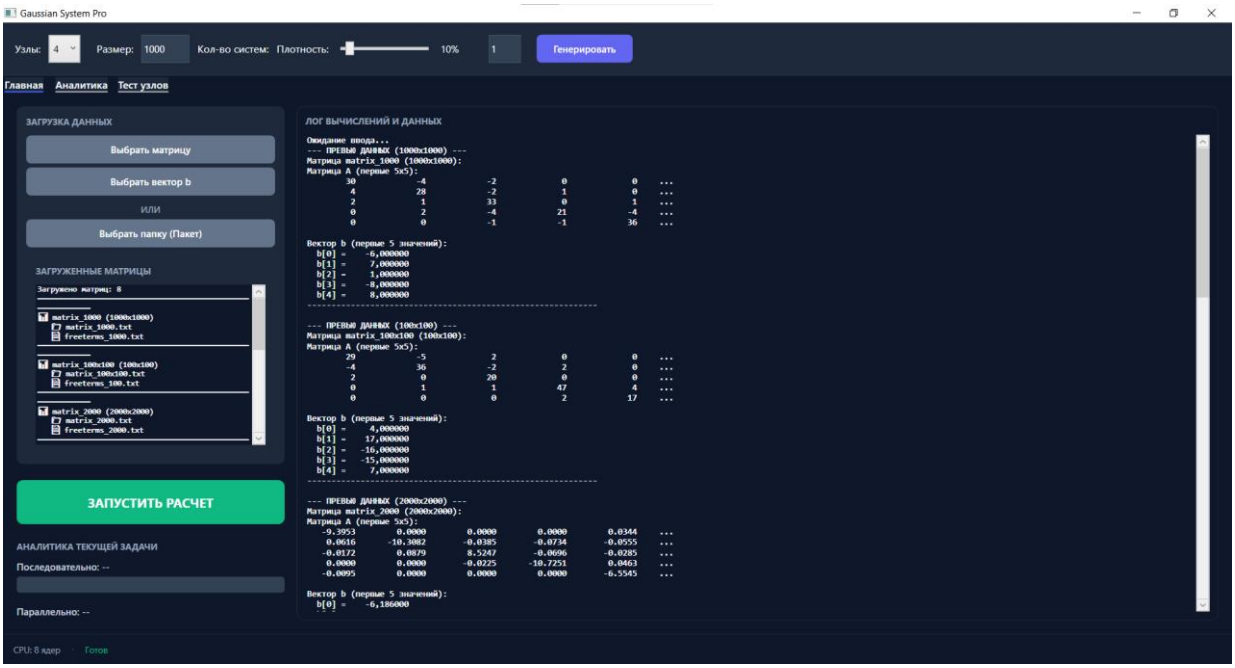


Рисунок Б.1 – Главное окно приложения

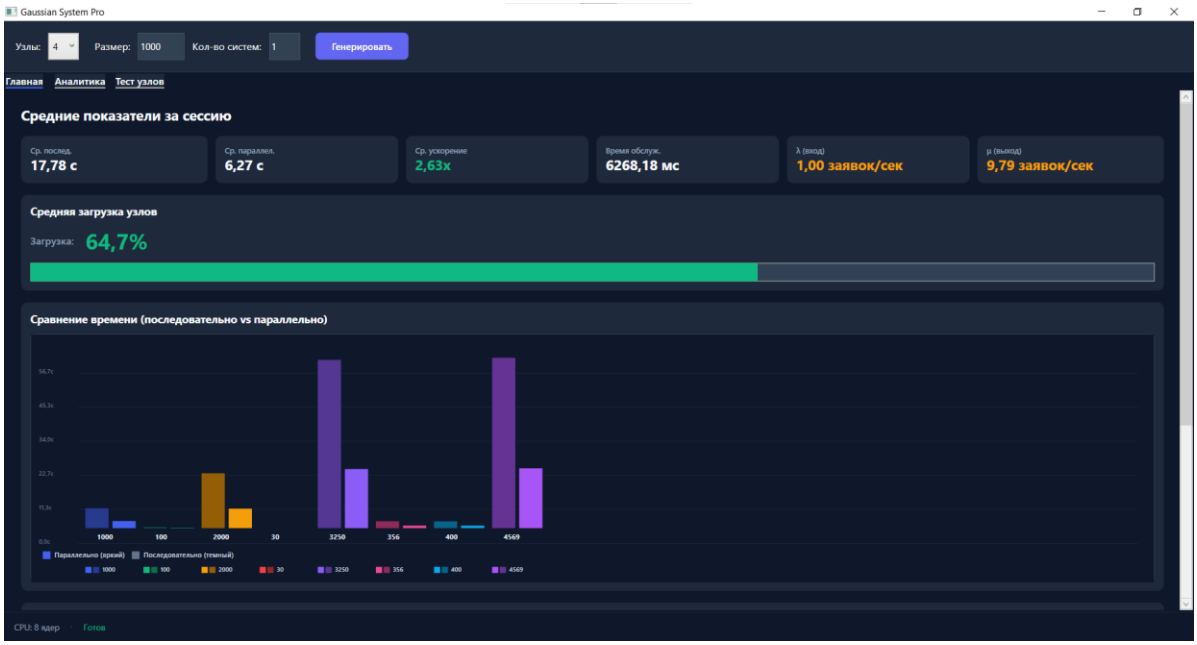


Рисунок Б.2 – Вкладка «Аналитика»

Рисунок Б.3 – Второй график на вкладке «Аналитика»

Рисунок Б.4 – Вкладка «Тест узлов»

ПРИЛОЖЕНИЕ В
(обязательное)

Листинг модульных тестов

DataGeneratorTests.cs

```
using Gaussian.Coordinator;
using Gaussian.Shared;
using Xunit;
using System.Linq;

namespace Gaussian.Tests;

public class DataGeneratorTests
{
    private readonly DataGenerator _generator = new();

    [Fact]
    public void GenerateRandomSystem_WithValidParameters_ReturnsMatrixAndVector()
    {
        int size = 10;
        double density = 0.3;

        var (matrix, vector) = _generator.GenerateRandomSystem(size, density);

        Assert.Equal(size, matrix.Size);
        Assert.Equal(size, vector.Length);
        Assert.NotNull(matrix.Rows);
    }

    [Fact]
    public void GenerateRandomSystem_WithFullDensity_ReturnsFullMatrix()
    {
        int size = 5;
        double density = 1.0;

        var (matrix, vector) = _generator.GenerateRandomSystem(size, density);

        for (int i = 0; i < size; i++)
        {
            Assert.Equal(size, matrix.Rows[i].Count);
        }
    }

    [Fact]
    public void GenerateRandomSystem_WithLowDensity_ReturnsSparseMatrix()
    {
        int size = 20;
        double density = 0.1;

        var (matrix, vector) = _generator.GenerateRandomSystem(size, density);
```

```
    int totalNonZero = 0;
    for (int i = 0; i < size; i++)
    {
        totalNonZero += matrix.Rows[i].Count;
    }

    double actualDensity = (double)totalNonZero / (size * size);
    Assert.True(actualDensity <= density + 0.15);
}

[Fact]
public void GenerateRandomSystem_WithZeroDensity_ReturnsOnlyDiagonal()
{
    int size = 10;
    double density = 0.0;

    var (matrix, vector) = _generator.GenerateRandomSystem(size, density);

    for (int i = 0; i < size; i++)
    {
        Assert.Contains(matrix.Rows[i], e => e.Column == i);
    }
}

[Fact]
public void GenerateRandomSystem_WithDifferentDensities_ReturnsDifferentSparsity()
{
    int size = 20;

    var (matrixLow, _) = _generator.GenerateRandomSystem(size, 0.1);
    var (matrixHigh, _) = _generator.GenerateRandomSystem(size, 0.5);

    int lowNonZero = 0;
    int highNonZero = 0;

    for (int i = 0; i < size; i++)
    {
        lowNonZero += matrixLow.Rows[i].Count;
        highNonZero += matrixHigh.Rows[i].Count;
    }

    Assert.True(lowNonZero < highNonZero);
}

[Fact]
public void GenerateRandomSystem_WithSize100_WorksWithoutError()
{
    var (matrix, vector) = _generator.GenerateRandomSystem(100, 0.2);

    Assert.Equal(100, matrix.Size);
    Assert.Equal(100, vector.Length);
```

```
}

[Fact]
public void GenerateRandomSystem_WithDensityZeroPointFive_ReturnsReasonableSparsity()
{
    int size = 50;
    double density = 0.5;

    var (matrix, vector) = _generator.GenerateRandomSystem(size, density);

    int totalElements = size * size;
    int nonZeroElements = 0;
    for (int i = 0; i < size; i++)
    {
        nonZeroElements += matrix.Rows[i].Count;
    }

    double actualDensity = (double)nonZeroElements / totalElements;
    Assert.InRange(actualDensity, 0.4, 0.6);
}

[Fact]
public void GenerateRandomSystem_ReturnsVectorOfCorrectSize()
{
    int size = 15;
    var (_, vector) = _generator.GenerateRandomSystem(size, 0.3);

    Assert.Equal(size, vector.Length);
}

[Fact]
public void GenerateRandomSystem_ReturnsNonEmptyRows()
{
    int size = 10;
    var (matrix, _) = _generator.GenerateRandomSystem(size, 0.2);

    for (int i = 0; i < size; i++)
    {
        Assert.NotEmpty(matrix.Rows[i]);
    }
}

[Fact]
public void GenerateRandomSystem_WithSize1_ReturnsSingleElement()
{
    var (matrix, vector) = _generator.GenerateRandomSystem(1, 0.5);

    Assert.Single(matrix.Rows);
    Assert.Single(vector);
    Assert.Single(matrix.Rows[0]);
}
```

```
[Fact]
public void GenerateRandomSystem_ValuesAreWithinExpectedRange()
{
    int size = 100;
    var (matrix, vector) = _generator.GenerateRandomSystem(size, 0.3);

    for (int i = 0; i < size; i++)
    {
        foreach (var el in matrix.Rows[i])
        {
            Assert.InRange(el.Value, -100, 100);
        }
        Assert.InRange(vector[i], -100, 100);
    }
}

[Fact]
public void GenerateRandomSystem_WithSameParameters_ReturnsDifferentSystems()
{
    int size = 10;
    double density = 0.3;

    var (matrix1, vector1) = _generator.GenerateRandomSystem(size, density);
    var (matrix2, vector2) = _generator.GenerateRandomSystem(size, density);

    bool hasDifference = false;
    for (int i = 0; i < size && !hasDifference; i++)
    {
        if (matrix1.Rows[i].Count != matrix2.Rows[i].Count)
            hasDifference = true;
        else if (vector1[i] != vector2[i])
            hasDifference = true;
    }
    Assert.True(hasDifference);
}

[Fact]
public void GenerateRandomSystem_HandlesLargeSize()
{
    var (matrix, vector) = _generator.GenerateRandomSystem(200, 0.1);

    Assert.Equal(200, matrix.Size);
    Assert.Equal(200, vector.Length);
}

[Fact]
public void GenerateRandomSystem_DiagonalElementAlwaysPresent()
{
    int size = 20;
    var (matrix, _) = _generator.GenerateRandomSystem(size, 0.05);

    for (int i = 0; i < size; i++)
```

```
        {
            Assert.Contains(matrix.Rows[i], e => e.Column == i);
        }
    }

    [Fact]
    public void GenerateRandomSystem_ColumnIndicesAreValid()
    {
        int size = 15;
        var (matrix, _) = _generator.GenerateRandomSystem(size, 0.3);

        for (int i = 0; i < size; i++)
        {
            foreach (var el in matrix.Rows[i])
            {
                Assert.InRange(el.Column, 0, size - 1);
            }
        }
    }
}
```

DataManagerTests.cs

```
using Gaussian.Coordinator;
using Gaussian.Shared;
using Xunit;
using System.IO;
using System.Linq;

namespace Gaussian.Tests;

public class DataManagerTests
{
    private readonly string _testDir;

    public DataManagerTests()
    {
        _testDir = Path.Combine(Path.GetTempPath(), "DataManagerTests_" +
Guid.NewGuid().ToString());
        Directory.CreateDirectory(_testDir);
    }

    private string CreateMatrixFile(double[][] data)
    {
        string path = Path.Combine(_testDir, "matrix.txt");
        var lines = data.Select(row => string.Join(" ", row));
        File.WriteAllLines(path, lines);
        return path;
    }

    private string CreateVectorFile(double[] data)
    {
        string path = Path.Combine(_testDir, "vector.txt");
```

```
        File.WriteAllLines(path, data.Select(x => x.ToString(System.Globalization.CultureInfo.Invari-
antCulture)));
        return path;
    }

    public void Dispose()
    {
        if (Directory.Exists(_testDir))
            Directory.Delete(_testDir, true);
    }

    [Fact]
    public void LoadFiles_WithValidFiles_ReturnsMatrixAndVector()
    {
        double[][] matrixData = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
        double[] vectorData = [10, 11, 12];

        string matrixPath = CreateMatrixFile(matrixData);
        string vectorPath = CreateVectorFile(vectorData);

        var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);

        Assert.Equal(3, matrix.Size);
        Assert.Equal(3, vector.Length);
        Assert.Equal(10, vector[0]);
        Assert.Equal(12, vector[2]);
    }

    [Fact]
    public void LoadFiles_WithSparseMatrix_HandlesZeros()
    {
        double[][] matrixData = [[1, 0, 3], [0, 5, 0], [7, 0, 9]];
        double[] vectorData = [10, 11, 12];

        string matrixPath = CreateMatrixFile(matrixData);
        string vectorPath = CreateVectorFile(vectorData);

        var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);

        Assert.Equal(2, matrix.Rows[0].Count);
        Assert.Equal(1, matrix.Rows[1].Count);
        Assert.Equal(2, matrix.Rows[2].Count);
        Assert.Equal(5, matrix.Rows[1][0].Value);
    }

    [Fact]
    public void LoadFiles_WithSpacesAndTabs_HandlesCorrectly()
    {
        string matrixPath = Path.Combine(_testDir, "matrix.txt");
        string vectorPath = Path.Combine(_testDir, "vector.txt");

        File.WriteAllText(matrixPath, "1\t2 3\n4 5\t6\n7\t8\t9");
```

```
    File.WriteAllText(vectorPath, "10\n11\n12");

    var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);

    Assert.Equal(3, matrix.Size);
    Assert.Equal(3, matrix.Rows[0].Count);
    Assert.Equal(3, matrix.Rows[1].Count);
    Assert.Equal(3, matrix.Rows[2].Count);
}

[Fact]
public void LoadFiles_WithDecimalNumbers_HandlesCorrectly()
{
    double[][] matrixData = [[1.5, 2.7, 3.9], [4.2, 5.3, 6.4]];
    double[] vectorData = [10.1, 11.2];

    string matrixPath = CreateMatrixFile(matrixData);
    string vectorPath = CreateVectorFile(vectorData);

    var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);

    Assert.Equal(2, matrix.Size);
    Assert.Equal(10.1, vector[0], 5);
    Assert.Equal(11.2, vector[1], 5);
}

[Fact]
public void LoadFiles_WithNegativeNumbers_HandlesCorrectly()
{
    double[][] matrixData = [[-1, -2, -3], [-4, -5, -6]];
    double[] vectorData = [-10, -11];

    string matrixPath = CreateMatrixFile(matrixData);
    string vectorPath = CreateVectorFile(vectorData);

    var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);

    Assert.Equal(-10, vector[0]);
    Assert.Equal(-11, vector[1]);
    Assert.Equal(-1, matrix.Rows[0][0].Value);
    Assert.Equal(-2, matrix.Rows[0][1].Value);
}

[Fact]
public void LoadFiles_WithEmptyLines_SkipsThem()
{
    string matrixPath = Path.Combine(_testDir, "matrix.txt");
    string vectorPath = Path.Combine(_testDir, "vector.txt");

    File.WriteAllText(matrixPath, "\n1 2\n\n3 4\n");
    File.WriteAllText(vectorPath, "5\n6");
```

```
    var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);

    Assert.Equal(2, matrix.Size);
    Assert.Equal(2, vector.Length);
}

[Fact]
public void LoadFiles_WithLargeMatrix_HandlesCorrectly()
{
    int size = 50;
    double[][] matrixData = new double[size][];
    for (int i = 0; i < size; i++)
    {
        matrixData[i] = Enumerable.Range(0, size).Select(j => (double)(i + j)).ToArray();
    }
    double[] vectorData = Enumerable.Range(0, size).Select(i => (double)i).ToArray();

    string matrixPath = CreateMatrixFile(matrixData);
    string vectorPath = CreateVectorFile(vectorData);

    var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);

    Assert.Equal(size, matrix.Size);
    Assert.Equal(size, vector.Length);
    Assert.Equal(0, vector[0]);
    Assert.Equal(size - 1, vector[size - 1]);
}

[Fact]
public void LoadFiles_WithMissingMatrixFile_ThrowsException()
{
    string vectorPath = CreateVectorFile([1, 2, 3]);

    Assert.Throws<FileNotFoundException>(() =>
        DataManager.LoadFiles(Path.Combine(_testDir, "missing.txt"), vectorPath));
}

[Fact]
public void LoadFiles_WithMissingVectorFile_ThrowsException()
{
    double[][] matrixData = [[1, 2], [3, 4]];
    string matrixPath = CreateMatrixFile(matrixData);

    Assert.Throws<FileNotFoundException>(() =>
        DataManager.LoadFiles(matrixPath, Path.Combine(_testDir, "missing.txt")));
}

[Fact]
public void LoadFiles_WithInvalidNumberFormat_ThrowsException()
{
    string matrixPath = Path.Combine(_testDir, "matrix.txt");
    string vectorPath = Path.Combine(_testDir, "vector.txt");
```

```
    File.WriteAllText(matrixPath, "a b c\n1 2 3");
    File.WriteAllText(vectorPath, "1\n2\n3");

    Assert.Throws<System.FormatException>(() =>
        DataManager.LoadFiles(matrixPath, vectorPath));
}

[Fact]
public void LoadFiles_WithEmptyMatrix_ReturnsEmptyRows()
{
    string matrixPath = Path.Combine(_testDir, "matrix.txt");
    string vectorPath = CreateVectorFile([1, 2, 3]);

    File.WriteAllText(matrixPath, "");

    var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);

    Assert.Equal(3, matrix.Size);
    for (int i = 0; i < 3; i++)
    {
        Assert.Empty(matrix.Rows[i]);
    }
}

[Fact]
public void LoadFiles_WithSingleElementMatrix_ReturnsSize1()
{
    double[][] matrixData = [[42]];
    double[] vectorData = [84];

    string matrixPath = CreateMatrixFile(matrixData);
    string vectorPath = CreateVectorFile(vectorData);

    var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);

    Assert.Equal(1, matrix.Size);
    Assert.Single(vector);
    Assert.Equal(42, matrix.Rows[0][0].Value);
    Assert.Equal(84, vector[0]);
}

[Fact]
public void LoadFiles_WithVerySmallValues_PreservesPrecision()
{
    double[][] matrixData = [[1e-10, 2e-10], [3e-10, 4e-10]];
    double[] vectorData = [5e-10, 6e-10];

    string matrixPath = CreateMatrixFile(matrixData);
    string vectorPath = CreateVectorFile(vectorData);

    var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);
```

```
        Assert.Equal(1e-10, matrix.Rows[0][0].Value, 15);
        Assert.Equal(5e-10, vector[0], 15);
    }

    [Fact]
    public void LoadFiles_WithVeryLargeValues_HandlesCorrectly()
    {
        double[][] matrixData = [[1e10, 2e10], [3e10, 4e10]];
        double[] vectorData = [5e10, 6e10];

        string matrixPath = CreateMatrixFile(matrixData);
        string vectorPath = CreateVectorFile(vectorData);

        var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);

        Assert.Equal(1e10, matrix.Rows[0][0].Value, 5);
        Assert.Equal(5e10, vector[0], 5);
    }

    [Fact]
    public void LoadFiles_FiltersOutNearZeroValues()
    {
        double[][] matrixData = [[1e-14, 5, 1e-13], [0, 0, 0]];
        double[] vectorData = [1, 2];

        string matrixPath = CreateMatrixFile(matrixData);
        string vectorPath = CreateVectorFile(vectorData);

        var (matrix, vector) = DataManager.LoadFiles(matrixPath, vectorPath);


        Assert.DoesNotContain(matrix.Rows[0], e => Math.Abs(e.Value) < 1e-14);
    }
}
```

ModelsTests.cs

```
using Gaussian.Shared;
using Xunit;
using System;

namespace Gaussian.Tests;

public class ModelsTests
{
    [Fact]
    public void SparseMatrix_Constructor_InitializesCorrectly()
    {
        int size = 5;
        var matrix = new SparseMatrix(size);

        Assert.Equal(size, matrix.Size);
```

```
    Assert.NotNull(matrix.Rows);
    Assert.Equal(size, matrix.Rows.Length);

    for (int i = 0; i < size; i++)
    {
        Assert.NotNull(matrix.Rows[i]);
        Assert.Empty(matrix.Rows[i]);
    }
}

[Fact]
public void SparseMatrix_Clone_ReturnsIndependentCopy()
{
    var original = new SparseMatrix(3);
    original.Rows[0].Add(new MatrixElement { Column = 1, Value = 2.5 });
    original.Rows[0].Add(new MatrixElement { Column = 2, Value = 3.7 });

    var clone = original.Clone();

    Assert.Equal(original.Size, clone.Size);
    Assert.Equal(original.Rows[0].Count, clone.Rows[0].Count);

    clone.Rows[0][0].Value = 999;
    clone.Rows[0].Add(new MatrixElement { Column = 0, Value = 100 });

    Assert.NotEqual(original.Rows[0][0].Value, clone.Rows[0][0].Value);
    Assert.NotEqual(original.Rows[0].Count, clone.Rows[0].Count);
}

[Fact]
public void MatrixElement_Properties_SetAndGet()
{
    var element = new MatrixElement { Column = 5, Value = 3.14159 };

    Assert.Equal(5, element.Column);
    Assert.Equal(3.14159, element.Value);
}

[Fact]
public void SolveRequest_Properties_SetAndGet()
{
    var request = new SolveRequest
    {
        MatrixPath = "matrix.txt",
        FreeTermsPath = "vector.txt"
    };

    Assert.Equal("matrix.txt", request.MatrixPath);
    Assert.Equal("vector.txt", request.FreeTermsPath);
}

[Fact]
```

```
public void SolveResponse_Properties_SetAndGet()
{
    var response = new SolveResponse();
    double[] solution = [1.0, 2.0, 3.0];

    response.Solution = solution;
    response.Size = 3;
    response.ParallelTimeMs = 100.5;
    response.SequentialTimeMs = 250.3;
    response.Acceleration = 2.49;
    response.Efficiency = 82.5;
    response.TheoreticalMax = 4.0;
    response.Cores = 8;
    response.MemoryUsageMb = 256.7;
    response.Residual = 1e-12;
    response.Log = "Test log";

    Assert.Equal(solution, response.Solution);
    Assert.Equal(3, response.Size);
    Assert.Equal(100.5, response.ParallelTimeMs);
    Assert.Equal(250.3, response.SequentialTimeMs);
    Assert.Equal(2.49, response.Acceleration);
    Assert.Equal(82.5, response.Efficiency);
    Assert.Equal(4.0, response.TheoreticalMax);
    Assert.Equal(8, response.Cores);
    Assert.Equal(256.7, response.MemoryUsageMb);
    Assert.Equal(1e-12, response.Residual);
    Assert.Equal("Test log", response.Log);
}

[Fact]
public void SolveResponse_DefaultValues_AreDefault()
{
    var response = new SolveResponse();

    Assert.Null(response.Solution);
    Assert.Equal(0, response.Size);
    Assert.Equal(0, response.ParallelTimeMs);
    Assert.Equal(0, response.SequentialTimeMs);
    Assert.Equal(0, response.Acceleration);
    Assert.Equal(0, response.Efficiency);
    Assert.Equal(0, response.TheoreticalMax);
    Assert.Equal(0, response.Cores);
    Assert.Equal(0, response.MemoryUsageMb);
    Assert.Equal(0, response.Residual);
    Assert.Equal("", response.Log);
}

[Fact]
public void AnalysisResult_Properties_SetAndGet()
{
    var result = new AnalysisResult
```

```
    {
        Size = 1000,
        SeqTime = 2500.5,
        ParTime = 625.1,
        Acceleration = 4.0,
        Efficiency = 80.0,
        TheoreticalMax = 8.0
    };

    Assert.Equal(1000, result.Size);
    Assert.Equal(2500.5, result.SeqTime);
    Assert.Equal(625.1, result.ParTime);
    Assert.Equal(4.0, result.Acceleration);
    Assert.Equal(80.0, result.Efficiency);
    Assert.Equal(8.0, result.TheoreticalMax);
}

[Fact]
public void SessionStats_Properties_SetAndGet()
{
    var now = DateTime.Now;
    var stats = new SessionStats
    {
        TaskSize = 1000,
        SequentialTimeMs = 5000,
        ParallelTimeMs = 1250,
        Acceleration = 4.0,
        Efficiency = 80.0,
        Timestamp = now,
        ArrivalRate = 1.5,
        ServiceRate = 2.0,
        ServiceTimeMs = 1250,
        NodeLoadPercent = 75.5,
        WorkersCount = 4,
        Residual = 1e-12
    };

    Assert.Equal(1000, stats.TaskSize);
    Assert.Equal(5000, stats.SequentialTimeMs);
    Assert.Equal(1250, stats.ParallelTimeMs);
    Assert.Equal(4.0, stats.Acceleration);
    Assert.Equal(80.0, stats.Efficiency);
    Assert.Equal(now, stats.Timestamp);
    Assert.Equal(1.5, stats.ArrivalRate);
    Assert.Equal(2.0, stats.ServiceRate);
    Assert.Equal(1250, stats.ServiceTimeMs);
    Assert.Equal(75.5, stats.NodeLoadPercent);
    Assert.Equal(4, stats.WorkersCount);
    Assert.Equal(1e-12, stats.Residual);
}

[Fact]
```

```
public void SolveTask_Properties_SetAndGet()
{
    var task = new SolveTask
    {
        MatrixPath = @"C:\matrix.txt",
        VectorPath = @"C:\vector.txt",
        Size = 500,
        Name = "TestMatrix"
    };

    Assert.Equal(@"C:\matrix.txt", task.MatrixPath);
    Assert.Equal(@"C:\vector.txt", task.VectorPath);
    Assert.Equal(500, task.Size);
    Assert.Equal("TestMatrix", task.Name);
}

[Fact]
public void WorkerBenchmark_Properties_SetAndGet()
{
    var benchmark = new WorkerBenchmark
    {
        WorkerCount = 8,
        TimeMs = 1234.56,
        MatrixSize = 2000,
        MatrixName = "LargeMatrix"
    };

    Assert.Equal(8, benchmark.WorkerCount);
    Assert.Equal(1234.56, benchmark.TimeMs);
    Assert.Equal(2000, benchmark.MatrixSize);
    Assert.Equal("LargeMatrix", benchmark.MatrixName);
}

[Fact]
public void SparseMatrix_WithZeroSize_InitializesEmptyRows()
{
    var matrix = new SparseMatrix(0);

    Assert.Equal(0, matrix.Size);
    Assert.NotNull(matrix.Rows);
    Assert.Empty(matrix.Rows);
}

[Fact]
public void SparseMatrix_AddElements_StoresCorrectly()
{
    var matrix = new SparseMatrix(3);

    matrix.Rows[0].Add(new MatrixElement { Column = 0, Value = 1 });
    matrix.Rows[0].Add(new MatrixElement { Column = 2, Value = 3 });
    matrix.Rows[1].Add(new MatrixElement { Column = 1, Value = 5 });
```

```
        Assert.Equal(2, matrix.Rows[0].Count);
        Assert.Single(matrix.Rows[1]);
        Assert.Empty(matrix.Rows[2]);
        Assert.Equal(1, matrix.Rows[0][0].Value);
        Assert.Equal(3, matrix.Rows[0][1].Value);
        Assert.Equal(5, matrix.Rows[1][0].Value);
    }

    [Fact]
    public void SessionStats_DefaultValues_AreDefault()
    {
        var stats = new SessionStats();

        Assert.Equal(0, stats.TaskSize);
        Assert.Equal(0, stats.SequentialTimeMs);
        Assert.Equal(0, stats.ParallelTimeMs);
        Assert.Equal(0, stats.Acceleration);
        Assert.Equal(0, stats.Efficiency);
        Assert.Equal(default, stats.Timestamp);
        Assert.Equal(0, stats.ArrivalRate);
        Assert.Equal(0, stats.ServiceRate);
        Assert.Equal(0, stats.ServiceTimeMs);
        Assert.Equal(0, stats.NodeLoadPercent);
        Assert.Equal(0, stats.WorkersCount);
        Assert.Equal(0, stats.Residual);
    }
}
```

SolutionCoordinatorTests.cs

```
using System;
using System.IO;
using System.Linq;
using Gaussian.Shared;
using Gaussian.Coordinator;
using Xunit;

namespace Gaussian.Tests
{
    public class SolutionCoordinatorTests : IDisposable
    {
        private readonly string _testDir;
        private readonly SolutionCoordinator _coordinator;

        public SolutionCoordinatorTests()
        {
            _testDir = Path.Combine(Path.GetTempPath(), "GaussianTests_" +
Guid.NewGuid().ToString());
            Directory.CreateDirectory(_testDir);
            _coordinator = new SolutionCoordinator();
        }

        public void Dispose()
```

```
    {
        _coordinator?.Shutdown();
        if (Directory.Exists(_testDir))
        {
            try { Directory.Delete(_testDir, true); }
            catch { }
        }
    }

    private (string matrixPath, string vectorPath) CreateTestFiles(int size, double diagValue = 5.0)
    {
        string matrixPath = Path.Combine(_testDir, $"matrix_{size}.txt");
        string vectorPath = Path.Combine(_testDir, $"vector_{size}.txt");

        var matrixLines = new string[size];
        for (int i = 0; i < size; i++)
        {
            var row = new double[size];
            row[i] = diagValue;
            matrixLines[i] = string.Join(" ", row.Select(x => x.ToString(System.Globalization.Cul-
tureInfo.InvariantCulture)));
        }
        File.WriteAllLines(matrixPath, matrixLines);

        var vectorLines = Enumerable.Repeat(diagValue.ToString(System.Globalization.Cul-
tureInfo.InvariantCulture), size).ToArray();
        File.WriteAllLines(vectorPath, vectorLines);

        return (matrixPath, vectorPath);
    }

    [Fact]
    public void Constructor_DoesNotThrow()
    {
        var coordinator = new SolutionCoordinator();
        Assert.NotNull(coordinator);
    }

    [Fact]
    public void Shutdown_WithNoWorkers_DoesNotThrow()
    {
        var exception = Record.Exception(() => _coordinator.Shutdown());
        Assert.Null(exception);
    }

    [Fact]
    public void Shutdown_MultipleCalls_DoesNotThrow()
    {
        _coordinator.Shutdown();
        _coordinator.Shutdown();
        _coordinator.Shutdown();
        Assert.True(true);
```

```
}

[Fact]
public void HandleRequest_WithZeroWorkers_UsesAtLeastOne()
{
    var (matrixPath, vectorPath) = CreateTestFiles(2);
    var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };

    var response = _coordinator.HandleRequest(request, 0);

    Assert.NotNull(response);
}

[Fact]
public void HandleRequest_WithNegativeWorkers_UsesOne()
{
    var (matrixPath, vectorPath) = CreateTestFiles(2);
    var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };

    var response = _coordinator.HandleRequest(request, -5);

    Assert.NotNull(response);
}

[Fact]
public void HandleRequest_ReturnsResponseWithSize()
{
    var (matrixPath, vectorPath) = CreateTestFiles(3);
    var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };

    var response = _coordinator.HandleRequest(request, 1);

    Assert.Equal(3, response.Size);
}

[Fact]
public void HandleRequest_ReturnsSequentialTime()
{
    var (matrixPath, vectorPath) = CreateTestFiles(2);
    var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };

    var response = _coordinator.HandleRequest(request, 1);

    Assert.True(response.SequentialTimeMs >= 0);
}

[Fact]
public void HandleRequest_ReturnsParallelTime()
{
    var (matrixPath, vectorPath) = CreateTestFiles(2);
    var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };
```

```
    var response = _coordinator.HandleRequest(request, 2);

    Assert.True(response.ParallelTimeMs >= 0);
}

[Fact]
public void HandleRequest_ReturnsCoresCount()
{
    var (matrixPath, vectorPath) = CreateTestFiles(2);
    var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };

    var response = _coordinator.HandleRequest(request, 1);

    Assert.Equal(Environment.ProcessorCount, response.Cores);
}

[Fact]
public void HandleRequest_ReturnsLogOkOnSuccess()
{
    var (matrixPath, vectorPath) = CreateTestFiles(2);
    var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };

    var response = _coordinator.HandleRequest(request, 1);

    Assert.Equal("OK", response.Log);
}

[Fact]
public void HandleRequest_WithMissingMatrix_ReturnsErrorLog()
{
    var request = new SolveRequest
    {
        MatrixPath = Path.Combine(_testDir, "missing.txt"),
        FreeTermsPath = Path.Combine(_testDir, "vector.txt")
    };
    File.WriteAllText(request.FreeTermsPath, "1.0\n2.0");

    var response = _coordinator.HandleRequest(request, 1);

    Assert.Contains("ERROR", response.Log);
}

[Fact]
public void HandleRequest_WithMissingVector_ReturnsErrorLog()
{
    var request = new SolveRequest
    {
        MatrixPath = Path.Combine(_testDir, "matrix.txt"),
        FreeTermsPath = Path.Combine(_testDir, "missing.txt")
    };
    File.WriteAllText(request.MatrixPath, "1 2\n3 4");
```

```
    var response = _coordinator.HandleRequest(request, 1);

    Assert.Contains("ERROR", response.Log);
}

[Fact]
public void HandleRequest_WithInvalidMatrix_ReturnsErrorLog()
{
    var (_, vectorPath) = CreateTestFiles(2);
    string matrixPath = Path.Combine(_testDir, "invalid.txt");
    File.WriteAllText(matrixPath, "a b c\nd e f");

    var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };

    var response = _coordinator.HandleRequest(request, 1);

    Assert.Contains("ERROR", response.Log);
}

[Fact]
public void HandleRequest_WithInvalidVector_ReturnsErrorLog()
{
    var (matrixPath, _) = CreateTestFiles(2);
    string vectorPath = Path.Combine(_testDir, "invalid.txt");
    File.WriteAllText(vectorPath, "a\nb");

    var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };

    var response = _coordinator.HandleRequest(request, 1);

    Assert.Contains("ERROR", response.Log);
}

[Fact]
public void HandleRequest_WithSize1Matrix_ReturnsSolution()
{
    string matrixPath = Path.Combine(_testDir, "size1.txt");
    string vectorPath = Path.Combine(_testDir, "vec1.txt");
    File.WriteAllText(matrixPath, "3.0");
    File.WriteAllText(vectorPath, "6.0");

    var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };
    var response = _coordinator.HandleRequest(request, 1);

    Assert.NotNull(response.Solution);
    Assert.Single(response.Solution);
    Assert.Equal(2.0, response.Solution[0], 5);
}

[Fact]
public void HandleRequest_WithSize2Diagonal_ReturnsCorrectSolution()
{
```

```
            var (matrixPath, vectorPath) = CreateTestFiles(2, 4.0);
            var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };
            var response = _coordinator.HandleRequest(request, 1);

            Assert.NotNull(response.Solution);
            Assert.Equal(2, response.Solution.Length);
            Assert.Equal(1.0, response.Solution[0], 5);
            Assert.Equal(1.0, response.Solution[1], 5);
        }

        [Fact]
        public void HandleRequest_ResidualIsSmall()
        {
            var (matrixPath, vectorPath) = CreateTestFiles(3, 5.0);
            var request = new SolveRequest { MatrixPath = matrixPath, FreeTermsPath = vectorPath };
            var response = _coordinator.HandleRequest(request, 1);

            Assert.True(response.Residual < 1e-10);
        }


    }
}
```

SolverTests.cs

```
using System;
using System.IO;
using System.Linq;
using System.Threading;
using Gaussian.Solver;
using Gaussian.Shared;
using Gaussian.Coordinator;
using Xunit;

namespace Gaussian.Tests
{
    public class SolverTests : IDisposable
    {
        private readonly DataGenerator _generator = new();
        private readonly GaussEngine _engine = new();
        private static readonly object _fileLock = new object();

        public SolverTests()
        {
            CleanCache();
        }

        public void Dispose()
        {
            CleanCache();
        }
```

```
private void CleanCache()
{
    lock (_fileLock)
    {
        string cacheFile = "gauss_cache.bin";
        for (int i = 0; i < 5; i++)
        {
            try
            {
                if (File.Exists(cacheFile))
                {
                    File.Delete(cacheFile);
                }
                break;
            }
            catch (IOException)
            {
                Thread.Sleep(100);
            }
            catch { }
        }
    }
}

private double ComputeResidual(SparseMatrix matrix, double[] b, double[] x)
{
    double maxResidual = 0;
    for (int i = 0; i < matrix.Size; i++)
    {
        double sum = 0;
        foreach (var el in matrix.Rows[i])
        {
            sum += el.Value * x[el.Column];
        }
        double residual = Math.Abs(sum - b[i]);
        if (residual > maxResidual) maxResidual = residual;
    }
    return maxResidual;
}

[Fact]
public void SolveSequential_With2x2System_ReturnsCorrectSolution()
{
    CleanCache();
    var engine = new GaussEngine();
    int n = 2;
    var matrix = new SparseMatrix(n);

    matrix.Rows[0].Add(new MatrixElement { Column = 0, Value = 2 });
    matrix.Rows[0].Add(new MatrixElement { Column = 1, Value = 3 });
    matrix.Rows[1].Add(new MatrixElement { Column = 0, Value = 5 });
    matrix.Rows[1].Add(new MatrixElement { Column = 1, Value = 7 });
```

```
    double[] b = [8, 19];

    double[] solution = engine.SolveSequential(matrix, b, out double ms);

    Assert.NotNull(solution);
    Assert.Equal(2, solution.Length);
    Assert.False(double.IsNaN(solution[0]));
    Assert.False(double.IsNaN(solution[1]));
    Assert.True(ms >= 0);
}

[Fact]
public void SolveSequential_WithIdentityMatrix_ReturnsSameVector()
{
    CleanCache();
    var engine = new GaussEngine();
    int n = 5;
    var matrix = new SparseMatrix(n);
    double[] expected = [1.5, 2.5, 3.5, 4.5, 5.5];

    for (int i = 0; i < n; i++)
    {
        matrix.Rows[i].Add(new MatrixElement { Column = i, Value = 1 });
    }

    double[] solution = engine.SolveSequential(matrix, expected, out double ms);

    Assert.NotNull(solution);
    Assert.Equal(n, solution.Length);
    for (int i = 0; i < n; i++)
    {
        Assert.Equal(expected[i], solution[i], 10);
    }
}

[Fact]
public void SolveSequential_WithDiagonalMatrix_ReturnsCorrectSolution()
{
    CleanCache();
    var engine = new GaussEngine();
    int n = 4;
    var matrix = new SparseMatrix(n);
    double[] expected = [2, 4, 6, 8];

    for (int i = 0; i < n; i++)
    {
        matrix.Rows[i].Add(new MatrixElement { Column = i, Value = i + 1 });
    }

    double[] b = expected.Select((val, i) => val * (i + 1)).ToArray();
```

```
    double[] solution = engine.SolveSequential(matrix, b, out double ms);

    Assert.NotNull(solution);
    Assert.Equal(n, solution.Length);
    for (int i = 0; i < n; i++)
    {
        Assert.Equal(expected[i], solution[i], 10);
    }
}

[Fact]
public void SolveSequential_With3x3System_ReturnsCorrectSolution()
{
    CleanCache();
    var engine = new GaussEngine();
    int n = 3;
    var matrix = new SparseMatrix(n);

    matrix.Rows[0].Add(new MatrixElement { Column = 0, Value = 1 });
    matrix.Rows[0].Add(new MatrixElement { Column = 1, Value = 2 });
    matrix.Rows[0].Add(new MatrixElement { Column = 2, Value = 3 });
    matrix.Rows[1].Add(new MatrixElement { Column = 0, Value = 4 });
    matrix.Rows[1].Add(new MatrixElement { Column = 1, Value = 5 });
    matrix.Rows[1].Add(new MatrixElement { Column = 2, Value = 6 });
    matrix.Rows[2].Add(new MatrixElement { Column = 0, Value = 7 });
    matrix.Rows[2].Add(new MatrixElement { Column = 1, Value = 8 });
    matrix.Rows[2].Add(new MatrixElement { Column = 2, Value = 9 });

    double[] b = [6, 15, 24];

    double[] solution = engine.SolveSequential(matrix, b, out double ms);

    Assert.NotNull(solution);
    Assert.Equal(3, solution.Length);
    Assert.False(double.IsNaN(solution[0]));
    Assert.False(double.IsNaN(solution[1]));
    Assert.False(double.IsNaN(solution[2]));
}

[Theory]
[InlineData(500, 0.3)]
[InlineData(1000, 0.3)]
[InlineData(2000, 0.3)]
public void SolveSequential_WithLargeMatrix_ReturnsSolution(int size, double density)
{
    CleanCache();
    var (matrix, vector) = _generator.GenerateRandomSystem(size, density);
    double[] solution = _engine.SolveSequential(matrix, vector, out double elapsedMs);

    Assert.NotNull(solution);
    Assert.Equal(size, solution.Length);
    Assert.All(solution, x => Assert.False(double.IsNaN(x)));
```

```
    Assert.True(elapsedMs > 0);

    double residual = ComputeResidual(matrix, vector, solution);
    Assert.True(residual < 1e-8);
}

[Theory]
[InlineData(500, 4)]
[InlineData(1000, 4)]
public void SolveParallel_WithLargeMatrix_ReturnsSameAsSequential(int size, int workers)
{
    CleanCache();
    var (matrix, vector) = _generator.GenerateRandomSystem(size, 0.3);
    double[] sequential = _engine.SolveSequential(matrix, vector, out _);
    double[] parallel = _engine.SolveParallel(matrix, vector, workers, out _);

    Assert.NotNull(sequential);
    Assert.NotNull(parallel);
    Assert.Equal(size, sequential.Length);
    Assert.Equal(size, parallel.Length);

    for (int i = 0; i < size; i++)
    {
        Assert.Equal(sequential[i], parallel[i], 10);
    }
}

[Fact]
public void SolveParallel_With2x2System_ReturnsSameAsSequential()
{
    CleanCache();
    var engine = new GaussEngine();
    int n = 2;
    var matrix = new SparseMatrix(n);

    matrix.Rows[0].Add(new MatrixElement { Column = 0, Value = 2 });
    matrix.Rows[0].Add(new MatrixElement { Column = 1, Value = 3 });
    matrix.Rows[1].Add(new MatrixElement { Column = 0, Value = 5 });
    matrix.Rows[1].Add(new MatrixElement { Column = 1, Value = 7 });

    double[] b = [8, 19];

    double[] sequential = engine.SolveSequential(matrix, b, out double seqMs);
    double[] parallel = engine.SolveParallel(matrix, b, 2, out double parMs);

    Assert.NotNull(sequential);
    Assert.NotNull(parallel);
    Assert.Equal(sequential[0], parallel[0], 10);
    Assert.Equal(sequential[1], parallel[1], 10);
}

[Fact]
```

```
        public void SolveParallel_WithSingleThread_EqualsSequential()
        {
            CleanCache();
            var engine = new GaussEngine();
            int n = 8;
            var matrix = new SparseMatrix(n);

            for (int i = 0; i < n; i++)
            {
                matrix.Rows[i].Add(new MatrixElement { Column = i, Value = 5.0 });
                if (i > 0)
                    matrix.Rows[i].Add(new MatrixElement { Column = i - 1, Value = 1.0 });
                if (i < n - 1)
                    matrix.Rows[i].Add(new MatrixElement { Column = i + 1, Value = 1.0 });
            }

            double[] b = new double[n];
            for (int i = 0; i < n; i++)
            {
                b[i] = 5.0;
                if (i > 0) b[i] += 1.0;
                if (i < n - 1) b[i] += 1.0;
            }

            double[] sequential = engine.SolveSequential(matrix, b, out double seqMs);
            double[] parallelSingle = engine.SolveParallel(matrix, b, 1, out double parMs);

            for (int i = 0; i < n; i++)
            {
                Assert.Equal(sequential[i], parallelSingle[i], 10);
            }
        }
    }
}
```

ПРИЛОЖЕНИЕ Г
(обязательное)

Результаты верификации приложения

```
=== РЕШЕНИЕ ДЛЯ 100x100 (первые 10) ===
x[0] = -0.030188
x[1] = -0.110610
x[2] = 0.010628
x[3] = 0.045239
x[4] = 0.103147
x[5] = 0.067829
x[6] = 0.078234
x[7] = -0.005674
x[8] = 0.012975
x[9] = 0.076097
```

sol100.txt – Блокнот

Файл Правка Формат Вид Справка

```
-0,030188
-0,110610
0,010628
0,045239
0,103147
0,067829
0,078234
-0,005674
0,012975
0,076097
```

```
=== РЕШЕНИЕ ДЛЯ 500x500 (первые 10) ===
x[0] = -0.004456
x[1] = -0.016003
x[2] = 0.002813
x[3] = -0.011749
x[4] = 0.001828
x[5] = -0.002311
x[6] = 0.002357
x[7] = -0.001641
x[8] = 0.012007
x[9] = 0.009259
```

sol500.txt – Блокнот

Файл Правка Формат Вид Справка

```
-0,004456
-0,016003
0,002813
-0,011749
0,001828
-0,002311
0,002357
-0,001641
0,012007
0,009259
```

```
=== РЕШЕНИЕ ДЛЯ 1000x1000 (первые 10)
x[0] = 0.006513
x[1] = 0.009146
x[2] = 0.000544
x[3] = -0.002903
x[4] = -0.005346
x[5] = -0.009622
x[6] = -0.008818
x[7] = -0.007684
x[8] = 0.002339
x[9] = 0.007357
```

sol1000.txt – Блокнот

Файл Правка Формат Вид Справка

```
0,006513
0,009146
0,000544
-0,002903
-0,005346
-0,009622
-0,008818
-0,007684
0,002339
0,007357
```

Рисунок Г.1 – Сравнение решений систем

Для системы 100x100

```
octave:164> x_sys100 = dlmread('sol100.txt', '\n');
x_oct100 = A100 \ b100;
err_abs100 = max(abs(x_sys100 - x_oct100));
err_rel100 = norm(x_sys100 - x_oct100) / norm(x_oct100);
fprintf('100x100: макс.погр=%e, отн.погр=%e\n', err_abs100,
err_rel100);
100x100: макс.погр=4.820594e-07, отн.погр=4.469003e-06
```

Для системы 500x500

```
octave:169> x_sys500 = dlmread('sol500.txt', '\n');
x_oct500 = A500 \ b500;
err_abs500 = max(abs(x_sys500 - x_oct500));
err_rel500 = norm(x_sys500 - x_oct500) / norm(x_oct500);
fprintf('500x500: макс.погр=%e, отн.погр=%e\n', err_abs500,
err_rel500);
500x500: макс.погр=4.978349e-07, отн.погр=2.351591e-05
```

Для системы 1000x1000

```
octave:174> x_sys1000 = dlmread('sol1000.txt', '\n');
x_oct1000 = A1000 \ b1000;
err_abs1000 = max(abs(x_sys1000 - x_oct1000));
err_rel1000 = norm(x_sys1000 - x_oct1000) / norm(x_oct1000);
fprintf('1000x1000: макс.погр=%e, отн.погр=%e\n', err_abs1000,
err_rel1000);
1000x1000: макс.погр=4.992036e-07, отн.погр=4.879685e-05
```

Рисунок Г.2 – Результат вычисления абсолютной и относительной погрешностей

Для системы 100x100

```
octave:149> A100 = load('matrix_100x100.txt');
b100 = load('freeterms_100.txt');
x_oct100 = A100 \ b100;
res_oct100 = norm(A100 * x_oct100 - b100);
fprintf('Octave невязка: %e\n', res_oct100);
Octave невязка: 5.208739e-14
```

```
>>> ОБРАБОТКА: matrix_100x100 (100x100)...
    Размерность: 100
    Файл матрицы: matrix_100x100.txt
    Файл вектора: freeterms_100.txt

[РЕЗУЛЬТАТЫ]
    Невязка: 6,288E-014
```

Для системы 500x500

```
octave:154> A500 = load('matrix_500x500.txt');
b500 = load('freeterms_500.txt');
x_oct500 = A500 \ b500;
res_oct500 = norm(A500 * x_oct500 - b500);
fprintf('Octave невязка 500x500: %e\n', res_oct500);
Octave невязка 500x500: 1.880348e-13
```

```
>>> ОБРАБОТКА: matrix_500x500 (500x500)...
    Размерность: 500
    Файл матрицы: matrix_500x500.txt
    Файл вектора: freeterms_500.txt

[РЕЗУЛЬТАТЫ]
    Невязка: 2,683E-013
```

Для системы 1000x1000

```
octave:159> A1000 = load('matrix_1000x1000.txt');
b1000 = load('freeterms_1000.txt');
x_oct1000 = A1000 \ b1000;
res_oct1000 = norm(A1000 * x_oct1000 - b1000);
fprintf('Octave невязка 1000x1000: %e\n', res_oct1000);
Octave невязка 1000x1000: 3.881534e-13
```

```
>>> ОБРАБОТКА: matrix_1000x1000 (1000x1000)...
    Размерность: 1000
    Файл матрицы: matrix_1000x1000.txt
    Файл вектора: freeterms_1000.txt

[РЕЗУЛЬТАТЫ]
    Невязка: 5,984E-013
```

Рисунок Г.3 – Сравнение коэффициентов невязок